Architectural Design and Evaluation of a Peer-to-Peer Solar Energy Trading System via Smart Contracts on a Simulated Solana-compatible Consortium Network

Chanthawat Kiriyadee
Computer Engineering and Artificial Intelligence
University of the Thai Chamber of Commerce
Bangkok, Thailand
2410717302003@live4.utcc.ac.th

บทคัดย่อ (Abstract) — The growing adoption of rooftop solar generation leaves many electricity users with surplus energy that they wish to trade directly. Yet direct Peer-to-Peer (P2P) electricity trading remains difficult, owing to grid constraints and the lack of a transparent price-formation mechanism. This paper presents the architectural design and evaluation of a P2P energy-trading system on a permissioned, governable consortium blockchain with predictable transaction cost. The core of the design is a clear separation of data verification from transaction settlement: off-chain verification is performed by an Aggregator Bridge that checks the Ed25519 signatures of meter readings before orders are matched by a Continuous Double Auction (CDA), while on-chain settlement records only verified matches and enforces correctness conditions at every step. The value of the work lies not in any single component but in the integration of all of them. Evaluation on a simulated system is consistent with this design: the ingest path sustains 80 meters at the design rate of 5.33 readings/s with no data loss, and under a step load up to 640 meters it keeps the loss ratio below 0.03%, with the measured ingest rate being a lower bound of a single sender client. The matching engine processes about 3.1×10^4 order pairs per second, and the single-pair on-chain settlement instruction costs about 97,000 compute units (80,000–103,000 across token-leg variants), roughly 48% of the default per-instruction budget. Meanwhile the settlement path is global-write-bound by design — every settlement must serialize on a central reconciliation write — which on a single validator yields a sub-1 transaction-per-second rate (about 0.5 TPS in a single recorded run, pending repeated measurement); even this conservative bound still supports about 450 settlements per 900-second clearing window, roughly an order of magnitude above the ≤ 40 matched pairs the tested 80-meter workload generates. These results support using the blockchain as a thin settlement layer. This is an architectural evaluation on a simulation, not a measurement on a real power system.

คำสำคัญ (Index Terms) — Consortium Blockchain, Peer-to-Peer, Continuous Double Auction, Proof of Authority, Microservices

I. INTRODUCTION

The expansion of Renewable Energy (RE) has led many electricity users to shift from being purely consumers to being producers and consumers of energy at the same time (Prosumers). This change creates challenges for the traditional power grid, which was designed primarily for centralized distribution to end users. Research on Peer-to-Peer energy markets therefore focuses on trading mechanisms that preserve both the constraints of the grid and the reliability of the power system [1], [2], [3].

The transition toward Smart Grid systems in the Thai context, following the master plan for the development of Thailand's power grid, employs Advanced Metering Infrastructure (AMI), making energy production and consumption data more granular. Integrating Distributed Energy Resources (DER) such as rooftop solar photovoltaic systems, electric vehicle (EV) charging stations, and Battery Energy Storage Systems (BESS) must account for requirements on DER interconnection, Microgrid controllers, Islanding, and the constraints of the low-voltage grid [4], [5], [6].

This work makes three main contributions. First, a decoupled architecture design that clearly separates off-chain data verification through the Aggregator Bridge — namely Ed25519 signature verification of meter readings following the DLMS/COSEM standard and evaluation of grid stability conditions together with order matching via Continuous Double Auction (CDA) — from on-chain settlement. Second, the design of a trust boundary for settlement on an Anchor/Solana-compatible Proof of Authority (PoA) consortium network, where the blockchain verifies the Ed25519 signatures of both buyer and seller, prevents replay through an order nullifier and escrow state, while the energy-quantity conditions and oracle attestation are enforced in the off-chain layer, making the blockchain act clearly as a settlement and audit layer (see หัวข้อ IV). Third, the development of an AMI simulation suite that relies on grid modeling with pandapower and a Smart Meter Simulator to generate deterministic meter readings, together with a preliminary measurement of the ingest rate of the telemetry ingest path. The main distinction from prior work (see หัวข้อ II) is not in any single component (consortium blockchain, CDA, or off-chain oracle, all of which exist in prior work), but in integrating these components

through a clearly defined trust boundary: a zone-partitioned CDA over a consortium PoA network that clearly separates the off-chain telemetry verification layer (Ed25519/DLMS) from the settlement layer. The on-chain settlement mechanism at the core of this work is atomic delivery-versus-payment (DvP) that combines the Ed25519 signature verification of both parties, partial-fill replay prevention through an order nullifier, and a cross-zone flow cap into a single set of correctness conditions enforced within a single transaction, which prior work has not specified systematically. The scope of this work is an architectural design and evaluation on a simulation system, not a measurement from a field power grid or a production Solana network. The empirical contribution of this work is therefore a reproducible single-system characterization — namely the ingest rate, compute-unit cost, and throughput of the settlement path — rather than a head-to-head quantitative comparison with other platforms, which is future work.

On the economic-mechanism side, the system uses an energy token issued from actual production and certified by a Renewable Energy Certificate (REC; the on-chain system uses the identifier erc), together with a stablecoin pegged to the Thai baht (THBC) for pricing and settlement, and the GRX token for staking and governance. The details of the pricing model are described in หัวข้อ V, and the details of the relevant programs are described in หัวข้อ VI.E.

This paper is organized as follows. หัวข้อ II reviews related research on blockchain and Peer-to-Peer energy markets. หัวข้อ III defines the system, trust, and attacker models. หัวข้อ IV describes the system's settlement model. หัวข้อ V describes the CDA pricing mechanism and the settlement computation. หัวข้อ VI describes the architecture, connectivity, and key implementation details. หัวข้อ VII specifies the experimental details and workload. หัวข้อ VIII summarizes the architectural evaluation, and หัวข้อ IX discusses results, limitations, and future development directions for the system. The acronyms frequently used in this paper are summarized in ตารางที่ I.

ตารางที่ I
Abbreviations used in this paper.

Acronym	Meaning
AMI	Advanced Metering Infrastructure (โครงสร้างพื้นฐานมาตรวัดขั้นสูง)
BESS	Battery Energy Storage System
BFT	Byzantine Fault Tolerance
CDA	Continuous Double Auction (ตลาดประมูลสองทางแบบต่อเนื่อง)
CPI	Cross-Program Invocation
CU	Compute Unit (หน่วยประมวลผลบนเชนของ Solana)
DAO	Decentralized Autonomous Organization
DER	Distributed Energy Resource (ทรัพยากรพลังงานแบบกระจายศูนย์)
DLMS/COSEM	Device Language Message Specification / COSEM (IEC 62056)
DSO	Distribution System Operator
DvP	Delivery-versus-Payment (การส่งมอบพร้อมชำระเงินแบบ atomic)
GRX / THBC	Governance token / THB-pegged stablecoin (โทเคนกำกับดูแล / stablecoin ตรึงเงินบาท)
IAM	Identity and Access Management
mTLS	Mutual TLS
OPF	Optimal Power Flow
PDA	Program Derived Address
PoA	Proof of Authority
PoH	Proof of History
RBAC	Role-Based Access Control
REC	Renewable Energy Certificate
SPIFFE	Secure Production Identity Framework for Everyone
SPL	Solana Program Library (Token)
SVM	Solana Virtual Machine
VPP	Virtual Power Plant

II. RELATED WORK

Blockchain technology originated with Bitcoin [7] as a decentralized ledger system that requires no intermediary, and was extended to Smart Contract execution with Ethereum [8], [9]. An overview of the technology and the classification of networks into permissionless and permissioned types was summarized by NIST [10], which serves as a reference framework for selecting a network architecture suited to governance requirements.

In the context of Peer-to-Peer energy markets, a large body of research has studied market mechanisms and auction design. Mengelkamp et al. [11] compared local energy market designs and bidding strategies, while Munsing et al. [12] proposed using blockchain for decentralized optimization of energy resources in microgrid networks. These works highlight the potential of blockchain to support direct energy trading between users, but most still evaluate on public networks that are constrained by transaction cost and block-confirmation latency. A recent concrete example is the decentralized trading platform of Esmat et al. [13], which develops a market mechanism and settlement on a public blockchain (Ethereum) but does not clearly separate the off-chain data-verification layer from on-chain settlement.

To address governance and cost predictability, a number of studies have turned to consortium or permissioned networks. Kang et al. [14] used a consortium blockchain for Peer-to-Peer electricity trading among plug-in electric vehicles, and Hyperledger Fabric [15] is an example of a distributed operating system for permissioned networks with clearly defined participant permissions. On the consensus side, the Byzantine Generals problem [16] and the Practical Byzantine Fault Tolerance algorithm [17] form the theoretical foundation for transaction confirmation in networks with a limited number of nodes, consistent with the permissioned-network assumption that governs validator admission via Proof of Authority (PoA) in this work (here PoA is an admission/governance layer, while Layer 1 consensus still relies on PoH together with Tower BFT).

The systematic review of Andoni et al. [18] comprehensively surveys blockchain applications in the energy sector and identifies scalability, cost, and governance as the main challenges, providing a foundational reference frame for this body of research. Recent literature reviews further reflect that this remains an open research area. Tanis et al. [19] comprehensively reviewed the market structure, operational layers, and multi-energy systems of Peer-to-Peer trading, while Bhavana et al. [20] surveyed blockchain applications in energy markets and green hydrogen supply chains, pointing out that scalability, cost, and regulatory compliance remain the main challenges. In addition, a comparative evaluation of consensus mechanisms for Peer-to-Peer energy trading in microgrids [21] supports the choice of a permissioned network with controlled permissions, consistent with the PoA-based admission/governance assumption used in this work.

Unlike the works above, this article focuses on the design and evaluation of the architecture of a simulation system that clearly separates off-chain verification through the Aggregator Bridge from the settlement layer on the blockchain. The scope of this work is therefore architectural design and evaluation, not the reporting of measurements from a field electrical grid or a Solana production network; the energy data used in the prototype comes from an AMI Simulator, and what is recorded on the blockchain is a settlement event that has already been verified by the Backend/Aggregator

Bridge, consistent with the preliminary guidance for testing microgrid control systems [22]. The positioning of this work relative to works that use blockchain for Peer-to-Peer energy markets is summarized in ตารางที่ II, where the main distinction is the integration of zone-partitioned CDA on a PoA-based consortium network with the clear separation of the off-chain meter-telemetry verification layer (Ed25519/DLMS) from the settlement layer. Note that ตารางที่ II is a qualitative positioning, since these works run on different networks, datasets, and assumptions, and therefore share no quantitative metric that is directly comparable.

ตารางที่ II
Positioning of this work vs. prior blockchain-based P2P energy-trading studies.

Work	Network	Consensus	Market mechanism	Off/on-chain separation
Mengelkamp [11]	Public (LEM)	Public	Local market / bidding	Not clearly separated
Munsing [12]	Public	Public	Decentralized optimization	Not clearly separated
Kang [14]	Consortium	Consortium	Iterative double auction	Partial
Hyperledger Fabric [15]	Permissioned	Pluggable (Raft/ BFT)	Platform (not market-specific)	—
Esmat [13]	Public (Ethereum)	Public	Double auction	Partial
This work	**Consortium (Solana-compatible)**	**PoH + Tower BFT (PoA admission/ governance)**	**CDA price-time zone-partitioned**	**Clearly separated + Ed25519/DLMS telemetry**

III. SYSTEM MODEL AND THREAT MODEL

This section defines the system model, the actors, the trust assumptions, and the adversary model of the simulated system, so that the security boundaries of the layered architecture are clear before the discussion of the settlement model in หัวข้อ IV. Since the scope of this work is a design on a simulated system, the analysis below states, in a straightforward manner, both the threats that the current mechanisms mitigate and the trust that still remains (residual trust).

A. System Model and Actors

The system comprises the following principal actors:

- Smart Meter (edge device): an endpoint device that holds a per-device Ed25519 key pair and signs energy readings before transmitting them.
- Aggregator Bridge: the off-chain validation and aggregation layer. It verifies the Ed25519 signature of every reading against the device's public key, decrypts the payload according to the DLMS/COSEM standard, and evaluates the surplus-energy and grid-stability conditions.
- Trading Service: matches buy/sell orders using a Continuous Double Auction (CDA).
- Chain Bridge: acts as the Reference Monitor and is the only service that holds the signing key and calls Solana RPC directly. Every transaction written to the chain is forced through a single signing path.
- Anchor Programs: the on-chain rule-enforcement layer that accepts only already-validated order pairs and serves as the settlement and audit layer.
- PoA / Governance Authority: the principal policy authority of the consortium that controls REC certification, authority permissions, and the aggregator allow-list.

B. Trust Assumptions

The system's trust assumptions are enforced across three boundaries: the device-to-bridge boundary, the service-to-service boundary, and the service-to-chain boundary.

At the device-to-bridge boundary, each device's identity is bound to a registered Ed25519 public key. The Aggregator Bridge verifies the signature of every reading (64-byte signature, 32-byte public key), both per-reading and in batch, using a fail-closed policy: when a key is not found or the key store is unresponsive, the system rejects (errors) rather than implicitly accepting the reading. The binary DLMS/ COSEM payload is encrypted with AES-256-GCM using the per-device key. In production mode, if a device has no encryption key, that frame is skipped (fail-closed), and the plaintext path is enabled only in development mode through an explicit configuration.

At the service-to-service boundary for chain writes (in particular the path to the Chain Bridge), communication uses ConnectRPC over mutually authenticated TLS (mTLS), binding service identity with a SPIFFE [23] X.509 identity verified from the client-side certificate during the handshake. Access permissions are derived from the SPIFFE URI of the verified certificate, mapped to a service role such as $\mathit{AggregatorBridge}$, $\mathit{TradingMatcher}$, or $\mathit{SettlementService}$. Identity is considered from the transport layer (L4), not from application headers (L7); an unverified caller is assigned the role $\mathit{Unknown}$. Note that the meter-data ingest path at the Aggregator Bridge uses request-level authentication via an API key (verified through IAM with a static key fallback) combined with the per-reading Ed25519 signature, not mTLS/SPIFFE. The authentication bypass (insecure mode) that grants full permissions is intended for development mode only.

At the service-to-chain boundary, the Chain Bridge is the only service that holds the key and submits transactions. The signing key is stored in HashiCorp Vault Transit (Ed25519) and never enters process memory; signing is restricted to the explicitly authorized key name only. For the asynchronous write path over NATS JetStream [24], the publisher must sign the message envelope with an ECDSA P-256 signature over the canonical bytes of the envelope, which is verified against the public key in the publisher's client certificate. The Chain Bridge verifies the chain (certificate ⟶ CA ⟶ SPIFFE SAN ⟶ service identity ⟶ signature) before the RBAC step, and value-moving subjects, such as minting energy tokens, are always required to carry a signed envelope.

C. Adversary Model and Mitigations

The adversary model considers an attacker who can craft, intercept, or replay messages on the network and may attempt to impersonate a device or service identity, but cannot access the private keys stored in Vault or the per-device keys. The threats considered and their mitigating mechanisms are summarized in ตาราง ที่ III.

ตารางที่ III
Threats considered and the mechanism that mitigates each.

Threat	Description	Mitigation
T1 Forged/replayed reading	Forge or replay a meter reading	Per-reading Ed25519 signature verification (fail-closed) + order nullifier in the settlement layer
T2 Service impersonation	Impersonate a service in the mesh	mTLS + SPIFFE identity ⟶ service role; unverified callers get *Unknown*
T3 Forged publisher / mint	Submit a forged chain-write command over NATS	Signed envelope bound to the certificate; mint subjects require a signature
T4 Replayed on-chain order pair	Settle the same order pair twice	Order nullifier account (PDA) — settle only once
T5 Duplicate/over-authorized mint	Mint more tokens than actual production	Idempotency key *(meter_id, window)* + PDA + REC co-signing

D. Residual Trust and Out-of-Scope

The system retains residual trust that must be stated explicitly. First, the Aggregator Bridge and the governance/oracle authority attest to the surplus-energy and grid-stability conditions in the off-chain layer, and these conditions are not re-verified on-chain; users must therefore trust the correctness of this off-chain validation layer. Compromise or collusion of the Aggregator Bridge or the oracle authority is not yet prevented by cryptographic mechanisms in the current prototype; approaches to further reduce trust, such as multi-oracle attestation or cryptographic proof, are future work (see หัวข้อ IX). Second, the security of the consensus layer (PoH combined with Tower BFT) rests on the assumption of a permissioned network that controls validator participation via PoA, where validators are authorized entities behaving according to policy — an architectural assumption that has not yet been quantitatively evaluated. Third, the authentication bypasses for development mode (insecure mode and plaintext fallback) must be disabled in production; otherwise they would break all of the above trust assumptions.

IV. SETTLEMENT MODEL

A. Architecture Overview

This work partitions responsibility into two interconnected trust domains joined through a single point. The off-chain trust domain performs validation and matching: it comprises the metering layer (a Smart Meter Simulator following the AMI standard and DLMS/COSEM) that feeds data into the Aggregator Bridge to verify Ed25519 signatures and screen the data, before forwarding it to the Trading Service, which matches orders using the Continuous Double Auction (CDA) algorithm, with the IAM Service and Notification Service supporting identity and notification. The on-chain trust domain settles transactions and records evidence: it comprises a set of Anchor programs (registry, trading, oracle, energy-token, governance, and treasury) on a permissioned (consortium) Solana-compatible network that records state into PDA accounts and an event log.

The crossing from the off-chain domain to the on-chain domain occurs solely through the Chain Bridge, the only service that contacts Solana RPC directly. It receives write commands over NATS JetStream and serves state reads over ConnectRPC on a channel protected by mTLS. The on-chain programs are Anchor programs [25] running on the Solana Virtual Machine (SVM) [26] and use Program Derived Addresses (PDA) to create accounts whose ownership is deterministically verifiable. Separating the two domains in this way means that power-engineering validation and access control happen before a transaction is recorded, while the blockchain serves as a thin settlement and audit layer. The full details of the services, their connectivity, and the architecture diagram appear in หัวข้อ VI. This article is an architectural evaluation on a simulated system, not a measurement from a production Solana network.

B. Settlement Model

The settlement model separates responsibility into two layers with clearly distinct trust boundaries. The off-chain layer performs validation and matching: it comprises the Aggregator Bridge, which verifies the Ed25519 signatures of meter readings, checks the data format against the DLMS/COSEM standard, and evaluates available-surplus conditions together with grid stability; and the Trading Service, which matches buy/sell orders using the Continuous Double Auction (CDA) algorithm. The output of this layer is a validated order pair together with an order payload signed by both parties. The on-chain layer is an Anchor program [25] that acts as a thin settlement and audit layer, accepting only validated order pairs and recording an auditable result.

Before recording a settlement, the on-chain program enforces only conditions that are verifiable from the payload and account state, namely: account-ownership checks, the Ed25519 signatures of both buyer and seller on the order payload, double-submission prevention via the order nullifier account, and the validity of the escrow state. Conditions that require external data, such as available surplus and oracle attestation, are enforced in the off-chain layer before submission and are not re-checked on-chain. This division of responsibility means the blockchain need not directly trust external data, yet can confirm that every settlement originates from an order pair genuinely signed by both parties. All account structures use Program Derived Addresses (PDA) to isolate the state of each transaction and to verify ownership deterministically. The details of the programs and PDA accounts are described in หัวข้อ VI.E [27].

C. Atomic Delivery-versus-Payment

A matched order pair is settled through the *settle_offchain_match* instruction of the trading program, which operates as delivery-versus-payment (DvP) within a single transaction; that is, the transfer of funds and the transfer of energy happen together or not at all. If any transfer fails, the entire transaction is reverted, so there is no lingering state in which one party has received while the other has not. Before the transfer, the program checks the Ed25519 signatures of both the buyer (instruction sysvar index 0) and the seller (index 1) over a message that binds the order's key fields, namely order_id, user, energy_amount, price_per_kwh, side, zone_id, and expires_at, making it impossible to alter the amount or price after signing. It then checks the price-crossing condition, namely that the match price must lie within the range $p_s \le p^* \le p_b$, checks the side of both parties, and checks the expiry time.

Settlement is partial-fill: the order nullifier account (seed *[nullifier, user, order_id]*) stores the accumulated filled amount (filled_amount) instead of a boolean value, so an order can be filled multiple times up to the signed amount, but the total will not exceed that amount ($\mathrm{match} \le \mathrm{energy\_amount} - \mathrm{filled}$) on either side. For cross-zone transactions that use inter-zone transmission lines, the program additionally enforces a cap on the accumulated committed flow on-chain ($\mathrm{committed\_flow} + \mathrm{match} \le \mathrm{capacity}$), whereas transactions within the same zone are exempt because they do

not use inter-zone transmission lines. Once all conditions pass, the total value and the shares are computed as follows, with currency flowing from the buyer escrow to the fee/wheeling/loss collectors and the seller escrow, and energy (energy token) flowing from the seller escrow to the buyer escrow. All transfers out of the escrow accounts are signed by the market_authority PDA as the owner of the escrow accounts (signing the token-transfer CPI), while authorization of the settlement comes from the Ed25519 signatures of the buyer and seller as described above. Besides the single-match instruction, the program also has a batch settlement instruction (*batch_settle_offchain_match*) that can combine up to 4 pairs per transaction to reduce cost, using the very same set of validation conditions (Ed25519 signatures, order nullifier, price crossing, and the cross-zone flow cap), and binding accounts from the signed payload by checking the Program Derived Address (PDA) of every account passed in.

$$V = \text{match} \cdot p^* \tag{1.1}$$

$$\text{fee} = \lfloor V \cdot \varphi / 10000 \rfloor \tag{1.2}$$

$$\text{net}_s = V - \text{fee} - w - c_{\text{loss}} \tag{1.3}$$

This set of equations is the on-chain rounded-down integer (fixed-point) form of Equation 4.1 through Equation 4.3 in หัวข้อ V.A, where V is the total value leaving the buyer escrow, φ is the market fee (market_fee_bps), w is the wheeling charge, c_{loss} is the loss cost (see Equation 2), and net_s is the seller's net amount. All computations use checked arithmetic instead of clamping values; that is, the value multiplication rejects the overflow case, and the deduction of the seller's net will **reject** the transaction (require! with error *ChargesExceedValue*) when the sum of the fee, wheeling and loss exceeds the value V, instead of rounding the amount down to zero, together with a cap on the total network fees of no more than 20% of V. When the market is configured to settle in THBC, the system enforces recording the gross value through the *record_settlement* CPI to the treasury program, so that the settlement counter reconciles against the actual cash flow leaving escrow.

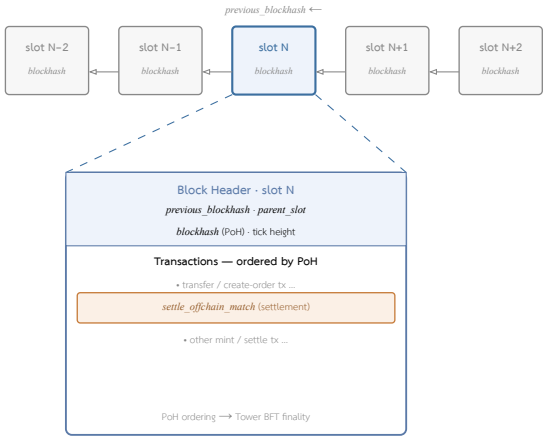
D. Consensus

The system's smart contracts are Anchor programs running on the Solana Virtual Machine (SVM). The ordering and consensus of transactions are therefore the responsibility of the Solana-compatible network on which the programs are deployed, and are not defined or tuned at the program level. The platform's consensus mechanism is split into two cooperating parts, namely Proof of History (PoH), a cryptographic verifiable clock that orders events in time before voting, and Tower BFT, a Byzantine Fault Tolerant voting mechanism developed from the Practical BFT (PBFT) [17] concept to confirm blocks and declare block finality [26].

In this work's evaluation, the Anchor programs are run on an SVM-level test environment (LiteSVM) and a single-node validator (localnet) to verify correctness and measure the compute-unit cost of the settlement path (see หัวข้อ VIII.E). Consequently, consensus properties such as leader selection, validator voting, and multi-node finality time are platform properties that are not measured in this work. The control of participation rights and consortium governance is a program-level governance layer separate from network-level consensus; details are in หัวข้อ VI.D.

E. Block and Settlement Transaction Structure

Because the programs run on a Solana-compatible network, the block structure and block header follow the platform's definition and are not tuned at the program level. Each block is bound to a slot with a target of approximately 400 milliseconds (see หัวข้อ VI.D). The order of transactions within is determined by the Proof of History (PoH) sequence before voting with Tower BFT, while each transaction references the latest blockhash to set its lifetime and prevent replay at the transaction level. These structures are platform properties [26], so this work emphasizes the structure of the settlement transactions that the program designs itself, rather than the block format. A block chain linked by previous blockhash is shown in รูปที่ 1.



รูปที่ 1. Block structure on a Solana-compatible network (defined by the platform, not tuned at the program level): a continuous chain of blocks (slot N−2 to N+2) in which each block references its predecessor through previous blockhash. Below, slot N is expanded; the block header contains the blockhash derived from PoH and parent_slot, while the transactions within, including the *settle_offchain_match* settlement instruction (see หัวข้อ IV.E), are ordered by Proof of History before finality is declared by Tower BFT.

A single settlement transaction comprises one settle instruction together with two Ed25519 signature-verification instructions per matched pair (for the buyer and the seller) that the program verifies through the instruction sysvar, where the signature payload (signature, public key and the canonical message, totaling about 189 bytes per instruction) resides in the instruction data, not in the accounts, so the address lookup table (ALT), which compresses only the account list, cannot reduce the size of this part. For this reason, even though the program permits batch settlement of up to 4 pairs per transaction (*batch_settle_offchain_match*), the practical cap is constrained by the 1,232-byte transaction packet size: combining two pairs (4 signature-verification instructions at about 760 bytes, together with the serialized BatchMatchPair at about 370 bytes, plus the account index list and header) already exceeds the packet size. Increasing the actual number of pairs per transaction therefore requires changing how the signatures are packed (for example, pre-verified signature accounts, or an off-chain aggregated multisig), not merely increasing the number of pairs. The accounts the transaction touches comprise the buyer's and seller's escrow accounts, the order nullifier accounts of both sides, the zone_market account, and the fee/wheeling/loss collector accounts in the treasury. This structural constraint explains why the batching cap and the throughput of settlement are tied to the transaction format, which is consistent with the measurement results in หัวข้อ VIII.F.

V. PRICING AND MARKET MECHANISM

A. Pricing and Settlement Model

The pricing model of this system is divided into three parts: clearing-price determination in the CDA mechanism, fee and seller-net computation at settlement, and the token pricing mechanism at the treasury layer. The symbols used in the equations are summarized in ตารางที่ IV.

ตารางที่ IV
Nomenclature for the pricing and settlement equations.

Symbol	Meaning
p_s	unit sell ask price
p_b	unit buy bid price
p^*	clearing price / landed cost
λ	loss factor ($\lambda \geq 1$)
c_{loss}	unit loss cost
w	wheeling charge per unit
m	incentive multiplier
δ	intra-zone discount
q	matched energy quantity (kWh)
V	total transaction value
f	market fee
φ	market fee rate (bps)
W	total wheeling charge ($q \cdot w$)
L	total loss cost ($q \cdot c_{\text{loss}}$)
r	exchange rate, GRX atoms per THBC
ψ	swap fee (bps)
A	reward accumulator per unit stake
s, s_{total}	staked amount and total staked amount
R	reward funded into the system

Clearing-price determination (CDA clearing) uses the seller-side price (maker price) adjusted by network costs. The per-unit loss cost is defined from the loss factor $\lambda \geq 1$ as in the equation

$$c_{\text{loss}} = p_s(\lambda - 1) \tag{2}$$

The clearing price, or landed cost, is computed from the sell ask price p_s plus the wheeling charge w and the loss cost, then adjusted by the incentive multiplier m and the intra-zone discount δ

$$p^* = (p_s + w + c_{\text{loss}}) \cdot m \cdot \delta \tag{3}$$

where $\delta = 0.95$ when buyer and seller are in the same zone, and $\delta = 1$ when crossing zones. An order can be matched when $p^* \leq p_b$ (where p_b is the buy bid price), and the system orders sellers with the lowest landed cost to be matched first under price-time priority. The matched quantity equals the smaller of the two sides' remaining amounts, $q = \min(q_b, q_s)$.

The fee and seller-net computation at settlement defines the total value, the market fee, and the seller's net amount as follows

$$V = q \cdot p^* \tag{4.1}$$

$$f = \frac{V \cdot \varphi}{10000} \tag{4.2}$$

$$\text{net} = V - f - W - L \tag{4.3}$$

where φ is the market fee in basis points (the on-chain default is 25 bps, or 0.25%), $W = q \cdot w$ is the total wheeling charge, and $L = q \cdot c_{\text{loss}}$ is the total loss cost. The amounts f, W, L, and net are transferred separately to the fee-collector, wheeling, loss, and seller accounts respectively. The zone parameters are interpreted in bps, namely $m = m_{\text{bps}}/10000$ and $w = w_{\text{bps}}/10000$, where the default wheeling charge equals 0 intra-zone and 0.02 cross-zone, while the loss factor equals 1.01 intra-zone and 1.03 cross-zone.

To illustrate the use of the equations above, consider an example of intra-zone matching with sell ask price $p_s = 4.00$ baht per kWh, quantity $q = 10$ kWh, and the intra-zone defaults ($\lambda = 1.01$, $w = 0$, $m = 1$, $\delta = 0.95$, $\varphi = 25$ bps). From Equation 2 we obtain $c_{\text{loss}} = 4.00(1.01 - 1) = 0.04$, and from Equation 3 the clearing price $p^* = (4.00 + 0 + 0.04) \cdot 1 \cdot 0.95 = 3.838$ baht per kWh, which can be matched when the buy bid price $p_b \geq 3.838$. The total value is then $V = 10 \cdot 3.838 = 38.38$ baht, the fee $f = 38.38 \cdot 25/10000 \approx 0.096$ baht, the total wheeling charge $W = 0$, and the total loss cost $L = 10 \cdot 0.04 = 0.40$ baht, giving the seller a net amount of $\text{net} = 38.38 - 0.096 - 0 - 0.40 \approx 37.88$ baht. The actual computation in the code uses floor-rounded fixed-point integers, so the resulting values may differ from this example at the rounding-fraction level.

The token pricing mechanism at the treasury layer covers the exchange between GRX and the THBC stablecoin pegged to the baht, using the rate r (number of GRX atoms per THBC) and the swap fee ψ (bps)

$$\text{thbc} = \frac{g \cdot r}{10^9} \cdot (1 - \psi/10000) \tag{5.1}$$

$$g = \frac{\text{thbc} \cdot 10^9}{r} \tag{5.2}$$

where redemption per Equation 5.2 incurs no fee, and the peg is maintained using a 1:1 supply-to-reserve condition, namely $\text{supply}_{\text{thbc}} \leq \text{reserve}_{\text{attested}}$. Staking rewards use a MasterChef-style accumulator, where a staker's accrued reward is computed from the staked amount s

$$\text{reward} = s \cdot A/10^{12} - \text{debt} \tag{6}$$

When a reward R is funded, the accumulator A is updated to $A \leftarrow A + (R \cdot 10^{12})/s_{\text{total}}$ pro-rata by stake share, and a slash deducts the requested amount but not more than the staked principal (capped at principal), then redistributes it back to the remaining stakers through the same accumulator.

On the production CDA path, the incentive multiplier is set to $m = 1$; that is, the incentive multiplier takes effect only on the feed-in or grid-export settlement path, not on CDA matching. In addition, the on-chain default of the market fee (25 bps) differs from the default in the off-chain configuration file (50 bps), and the zone parameters in the governance program are stored at ×1000 scale, while the consumer of the actual values in the trading layer interprets them in bps ($/10000$), which is the value the system actually uses. The computation in the code uses floor-division fixed-point integers, and the seller net uses checked arithmetic that rejects a transaction when the total fees exceed the matched value (with a network fee ceiling of 20%) instead of clamping the amount down to zero. Therefore the equations above constitute a real-value model that may differ from the actual computed result at the rounding-fraction level.

B. Sensitivity of Seller Net and P2P Trading Uplift

To show the economic implications of the pricing model above, this section analyzes the sensitivity of the seller's net amount to the zone parameters and the fee. This is a purely model-derived computation from the settlement equations in Equation 3 and Equation 4.3, not a measurement of revenue from running the real system. We fix the base sell ask price $p_s = 4.00$ baht per kWh, the matched quantity $q = 10$ kWh, and the incentive multiplier $m = 1$ (per the real CDA path), then vary the intra-zone/cross-zone state, the fee rate φ, and the loss factor λ as in ตารางที่ V.

ตารางที่ V

Model-derived seller-net sensitivity computed from Equation 3 and Equation 4.3 at $p_s = 4.00$ ฿/kWh, $q = 10$ kWh, $m = 1$. "net" is the seller's settled amount; wheeling (w) and loss (λ) are pass-through to the buyer via the landed price p^*.

Scenario	δ	w	λ	φ (bps)	p^*	net (฿)	net/kWh
S1 intra-zone	0.95	0	1.01	25	3.838	37.88	3.79
S2 cross-zone	1	0.02	1.03	25	4.14	39.9	3.99
S3 intra-zone, high fee	0.95	0	1.01	100	3.838	37.6	3.76
S4 cross-zone, high loss	1	0.02	1.05	25	4.22	39.89	3.99

From ตารางที่ V, two important patterns are visible. First, the seller's net amount barely changes when the wheeling charge w or the loss factor λ is increased (compare S2 with S4, which differ at the level of fractions of a satang), because both costs are added into the landed price p^* paid by the buyer and then split off to the wheeling-collector and loss accounts. They are therefore pass-through costs that do not directly reduce the seller's amount, so the seller always receives an amount close to $q \cdot p_s$. Second, the variables that significantly affect the seller's amount are the intra-zone discount δ and the fee rate φ: intra-zone matching ($\delta = 0.95$) reduces the seller's amount by about 5% to incentivize local consumption, while raising the fee from 25 to 100 bps (S1 $\longrightarrow$ S3) reduces the net amount only slightly.

Compared with a flat feed-in tariff for surplus power under a hypothetical citizen-solar program assumed at around 2.20 baht per kWh, the seller's per-unit net amount in the P2P market (approximately 3.76–3.99 baht per kWh) represents an uplift of about 72–81%. At the same time, the buyer still pays a landed price p^* (approximately 3.84–4.22 baht per kWh) lower than the usual retail electricity price, so gains from trade arise on both sides. Here the 2.20-baht purchase rate is merely a hypothetical reference value for comparison, not a value measured from a real market.

A limitation of this analysis is that the numbers in ตารางที่ V are model-derived results under fixed parameters and a single matched quantity (10 kWh). Moreover, the actual computation in the code uses floor-rounded fixed-point integers, so the resulting values may differ at the rounding-fraction level, and this is not yet a measurement of revenue distribution under a full-scale simulated workload, which is left as future work (see หัวข้อ IX).

C. Continuous Double Auction Matching

The market mechanism uses Continuous Double Auction (CDA) matching that orders by price-time priority and accounts for the topological constraints of the power network. Sell orders are partitioned into a zone-segmented order book, where each zone stores sell orders in an ordered structure (BTreeMap) keyed by *(price, created_at, id)*, so that the key ordering prioritizes the lowest price first, then the earlier order-creation time, and uses the order id as the final tie-breaker, thereby yielding price-time priority implicitly without re-sorting. Orders whose remaining quantity falls below the minimum threshold (MIN_TRADE_AMOUNT) or that have expired are not inserted into the book.

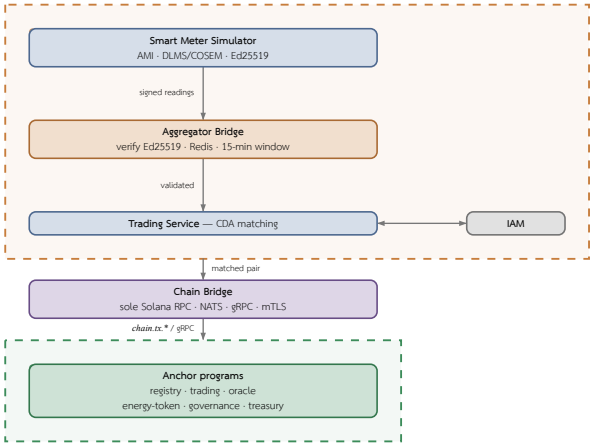
For each buy order, the matching engine gathers candidate sellers only from zones from which the network can deliver energy to the buyer's zone, via two-stage topology pre-filtering: the first stage checks at the minimum quantity to immediately exclude zones that cannot reach each other, and the second stage re-checks at the actual matched quantity (*can_accommodate_flow(sell_zone, buy_zone, amount)*) to enforce the transmission-line capacity ceiling. Within each reachable zone, a range query is used to retrieve only sell orders whose ask price does not exceed the bid price, then the landed cost is computed per Equation 3 (including wheeling, loss cost, multiplier, and the intra-zone discount $\delta = 0.95$). Only sell orders whose landed cost does not exceed the bid price ($p^* \leq p_b$) pass through as candidates. The system prevents self-trade by skipping pairs where the buyer and seller are the same user.

Once candidates from all reachable zones are obtained, the system consolidates the list and sorts by landed cost from low to high, so the buyer gets the cheapest landed total price first, regardless of which zone that sell order is in. It then matches progressively, with the per-pair quantity equal to the smaller remaining amount of the two sides ($q = \min(q_b, q_s)$) as a partial-fill, and immediately removes sell orders whose remaining quantity falls below the threshold from the book. For Fill-or-Kill (FOK) buy orders, the system first checks whether the total candidate quantity is sufficient for the entire order; if not, it does not match at all. In addition, there is match consolidation when the same buyer-seller pair and the same price occur consecutively, to reduce the number of settlement records that must be sent on-chain. Each match result records the match price (landed cost), wheeling charge, loss cost, and source/destination zones, before sending the matched pairs to atomic settlement per หัวข้อ IV. The on-chain processing cost of the settlement path fed by the matching engine is reported in หัวข้อ VIII.E, while the matching throughput of the matching engine in the in-memory layer is reported in หัวข้อ VIII.D.

VI. SYSTEM DESIGN AND IMPLEMENTATION

A. Architecture Overview

The system architecture is designed to support the simulation of Peer-to-Peer energy trading. This work presents a microgrid model that divides the data path into the Smart Meter, Aggregator Bridge, Trading Service, Anchor Smart Contract Program, Settlement Engine, and Frontend Dashboard layers. Energy generation and consumption data are produced by the Smart Meter Simulator and then sent into the Aggregator Bridge to be screened before being converted into transaction instructions for the Anchor program on a permissioned Solana-compatible network. The decomposition of the system into off-chain and on-chain domains, together with the primary data paths and the trust-boundary crossing point, is summarized in รูปที่ 2.



รูปที่ 2. System architecture and the off-chain/on-chain trust boundary. The off-chain domain (orange, dashed) verifies Ed25519-signed meter telemetry, aggregates it, and matches orders via CDA; the on-chain domain (green, dashed) settles and audits the verified pairs. Chain Bridge is the sole crossing between the two — the only service touching Solana RPC, writing via NATS ***chain.tx.**** and reading via gRPC over mTLS.

This decomposition ensures that electrical-engineering validation and access control occur before transactions are recorded, while the Anchor program acts as a verifiable rule-enforcement layer that records state for retrospective auditing. The Settlement Engine, in turn, manages signed transactions, tracks outcomes from the transaction signature, and reads the event log to bring state back for display on the Frontend.

B. Backend Services

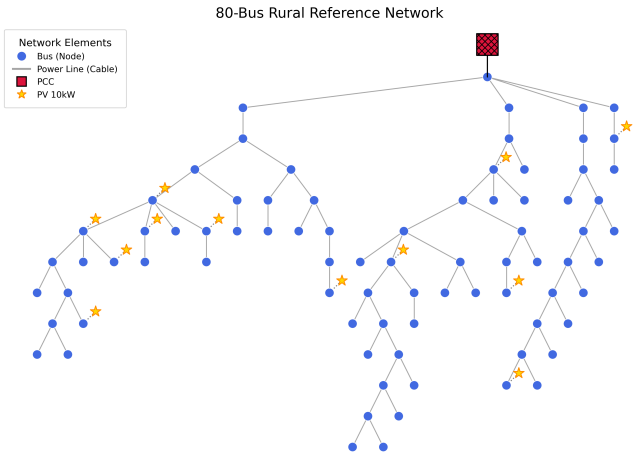
The Backend layer acts as the intermediary between the Smart Meter Simulator (see หัวข้อ VI.C), the Aggregator Bridge, and the Smart Contract. It is responsible for collecting data, validating correctness, queuing transactions, and dispatching instructions to the blockchain only after the data has passed the grid-stability conditions. The design adopts a Microservice approach to split the workload into smaller sub-workloads, allowing the system to scale only the parts with high request volume and to limit the impact when any single service fails. Details of the inter-service communication channels (ConnectRPC over mTLS and NATS JetStream) and the associated trust assumptions are described in หัวข้อ III. The main components of the Backend layer comprise:

- IAM Service: manages identity, on-chain data access rights, and user roles such as Prosumer, Consumer, and Operator.
- Aggregator Bridge: receives electricity generation and consumption data from the Smart Meter, validating the data format, reference time, device identifier, and energy values before forwarding them to the analysis stage.
- Trading Service: validates and matches energy trade orders together with grid-stability conditions such as remaining energy quantity, network constraints, and Smart Meter connection status.
- Chain Bridge: the sole intermediary that dispatches instructions to the Solana Program and tracks results from the transaction signature or event log; no other service calls Solana RPC directly.
- Notification Service: sends notifications, such as Email and Alerts, when a trade order or settlement status changes.

This separation of services helps the system better support large volumes of real-time Smart Meter data, since the data-ingestion service can scale its instance count without affecting the transaction-validation service or the blockchain-connection service. In addition, the Backend layer serves as the security control point before transactions are recorded to the Smart Contract, so that the system does not rely on the blockchain alone but integrates electrical-engineering validation with user access control.

C. Grid and Meter Simulation

The Grid Modeling is designed to simulate complex electrical systems, Distributed Energy Resources (DERs), and the operation of a Virtual Power Plant (VPP), thereby enabling highly accurate simulation of Advanced Metering Infrastructure (AMI) and grid management. The core of the simulation system integrates Physics-validated State Estimation in real time and Optimal Power Flow (OPF) using Pandapower, which enables the system to generate deterministic meter data for large electrical grids.



รูปที่ 3. Grid bus network topology used by the simulator.

The Grid Simulator is developed with Python 3.11 [28] to simulate the electricity-consumption behavior of a Distribution Grid. It uses the Network Topology format from GridLAB-D GLM files [29] and converts it into a grid model consisting of bus, line, load, and photovoltaic units, as shown in รูปที่ 3. The main tools used include FastAPI [30], NetworkX [31] for representing the topology structure, pvlib [32] for simulating photovoltaic generation, and pandapower [33] for computing power flow.

The simulation process operates in discrete time-steps, where each round consists of two stages: generating readings at the meter level (see หัวข้อ VI.C.1), followed by solving the network state at the grid level (see หัวข้อ VI.C.2). In addition to synthetic readings, the system can also replay real telemetry data from CSV files and associate the data with a bus through the meter registry, thereby supporting hybrid simulation between measured data and synthetic data. The correctness of the simulator is verified through a test suite covering topology loading and meter-to-bus mapping.

1) Smart Meter Simulator

For each bus in the topology, the system generates a meter population according to the proportions of user types, where each meter (SmartMeter) is composed of modular sub-device models, namely the Load, the Solar (photovoltaic) panel when installed, and an optional Battery Energy Storage System (BESS). Electricity-consumption behavior is simulated with a voltage-dependent ZIP load model, in which the real power drawn from the network depends on the per-unit voltage according to the equation

$$P(V) = P_{\text{base}}(Z \cdot V^2 + I \cdot V + P), \quad Z + I + P = 1 (7)$$

where the Z (impedance) component varies with V^2, the I (current) component varies with V, and the P (power) component is constant. The meters in this evaluation set the impedance fraction at 0.20 (*ZIP_IMPEDANCE_FRACTION=0.20*) and clamp the voltage into the range $[0, 1.5]$ per unit before computation. The generation from the photovoltaic panels is computed with pvlib [32] using the Ineichen clear-sky model together with the PVWatts model for direct-current power, then derated by a weather derate factor — for example, cloudy multiplied by 0.42 and partly cloudy multiplied by 0.72. If pvlib is unavailable, the system falls back to a functional generation profile as a backup.

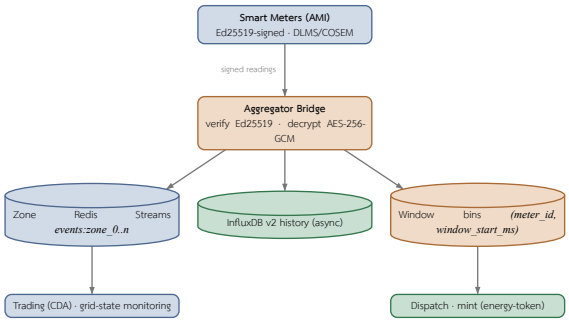
To make the simulation deterministic, each meter draws noise from its own dedicated stream by seeding its random number generator from the XOR between the system-wide seed and the SHA-256 digest of the meter identifier, so that adding or removing a single meter does not shift the readings of other meters. The result per round is an energy reading record (EnergyReading) that holds the energy produced, the energy consumed, the surplus energy, and the deficit energy as non-negative kWh values, together with the interval length of that round. Before export, each meter has a unique Ed25519 identity (per-meter key) generated deterministically from the SHA-256 of *secret:meter_id*, and signs a canonical string of the form *device_id:kwh:timestamp_ms*, exported as a base58 signature, allowing the Aggregator Bridge to verify the provenance of each reading point-by-point without keeping the meters' secret keys centrally.

2) Grid Simulation

In each time tick, once the readings of all meters have been collected, the system computes the net power injection of each bus from the difference between generation and consumption, then solves the power flow to update the network state, namely the per-unit voltage of each bus, the line flow, the power loss, and the line utilization. The main solver uses pandapower [33] with a backward/forward sweep (BFSW), which is an accurate AC power flow solver for radial distribution networks; and when pandapower is unavailable or the computation does not converge (non-convergence), the system falls back to an approximate DistFlow solver as a backup so that the simulation can continue.

In addition to solving the basic power flow, the grid simulation layer also simulates voltage-control mechanisms and abnormal events of the distribution network, namely the volt-watt and volt-VAR response of inverters, on-load tap changer (OLTC) adjustment, fault injection at a line, bus, or transformer, and tie-switch operation to transfer load, so that the dataset sent to the Aggregator Bridge reflects the physical behavior of the network under a variety of operating and abnormal conditions, rather than merely independently randomized energy values. That said, the above grid-physics capabilities (volt-VAR, OLTC, fault injection, and tie-switch) are part of the simulation suite but are not yet used as variables in the evaluation reported in this article, which measures only the ingest rate of signed readings, the on-chain settlement cost, and the matching rate (see หัวข้อ VIII). Evaluating the impact of these grid-stability conditions on matching and settlement is therefore left as future work.

To conform to industry standards and to certify data provenance, the Aggregator Bridge receives high-frequency real-time meter data and disseminates it through zone-partitioned Redis Streams for dynamic monitoring of network state, while aggregating readings into 15-minute time windows for use in the assessment of generation capacity and dispatch in the Backend layer. The meter data format follows the DLMS/COSEM standard (IEC 62056), decoding the binary payload portion with AES-256-GCM and re-publishing it in JSON form. In addition, the system guarantees cryptographic non-repudiation by verifying asymmetric Ed25519 cryptographic signatures, both for individual readings and in batches, before admitting data into the system. That said, in the current prototype there is not yet a path to construct an on-chain Settlement Attestation directly from the metering layer, which is an avenue left open for future work. The data-dissemination path of the Aggregator Bridge is summarized in รูปที่ 4.



รูปที่ 4. Aggregator Bridge telemetry dissemination: verified readings fan out to zone-partitioned Redis Streams (real-time), an InfluxDB history sink (async, fire-and-forget), and 15-min aggregation windows keyed by ***(meter_id, window_start_ms)***. Each sink sits above the consumer it feeds; the InfluxDB history sink is terminal.

D. Consortium Network

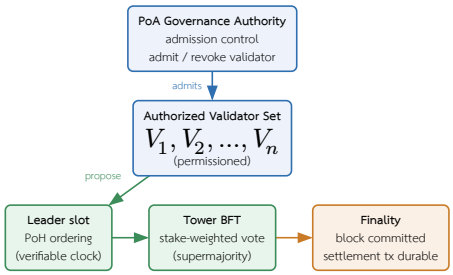
The blockchain network in this system is designed as a Consortium Network under the assumption of Proof of Authority (PoA) participation governance, as described in หัวข้อ IV; this section therefore focuses primarily on the details of the network design and its governance. The design ensures that block validators are entities that have a duty to verify, or have obtained consent from, the DSO — for example, a Load Aggregator, a regulatory body, or an authorized organization. The main reason for choosing PoA is network governance that is easier to control than public networks such as Solana Mainnet or Ethereum, and its suitability for regulatory-compliance requirements and cost predictability in experiments or deployments in energy systems that must estimate transaction costs in advance [15], [34]. The important point to emphasize is that PoA here is a governance and admission-control layer, not a replacement for the Layer 1 consensus mechanism of the Solana-compatible network, which still relies on Proof of History (PoH) together with Tower BFT for ordering and announcing block finality, as described in หัวข้อ IV.

Permission and governance policies are enforced through the on-chain governance program, which covers three main mechanisms. The first mechanism is governing the primary authorized entity (the PoA authority) through a two-step handover, where propose_authority_change lets the current authority set a pending authority together with an expiry time, and approve_authority_change must be called by the pending authority itself to accept the transfer — so that authority cannot be handed

to an unconsenting or erroneous account, and only one pending change request is allowed at a time. The second mechanism is controlling the allow-list of Aggregators authorized to sign readings, via admit_aggregator and revoke_aggregator. The third mechanism is DAO voting via create_proposal, cast_vote, and execute_proposal for adjusting the economic parameters of each zone (zone_config), namely the incentive multiplier, wheeling charge, loss factor, and maintenance mode.

The voting uses weighting by the cumulative generation of meters (stake-by-generation), where a voter's weight is computed as $w = \max\left(100, \frac{\text{total generation}}{1000}\right)$ — that is, every 1,000 kWh of cumulative generation is equivalent to one weight unit, with a minimum of 100 so that small participants still have a voice. Double-voting is prevented using a PDA vote record account per (proposal, voter) pair, one account per vote. When the voting period ends, execute_proposal tallies the result automatically: a proposal Passes only if the total votes reach the minimum quorum threshold (min_quorum_votes set in poa_config) and the votes in favor exceed the votes against; otherwise it is Rejected, thereby preventing parameter changes by too few voters.

The difference from the Solana public mainnet is that this network is not open to external validators joining permissionlessly, and it can define permission policies, program upgrades, and data retention according to the project's requirements. However, the execution layer still references the Solana Virtual Machine architecture and the Sealevel parallel runtime so that transactions not using the same account can be processed in parallel. The slot time near 400 ms and the compute budget mentioned in this article are design targets referenced from the Solana documentation, not measured results of the permissioned network. The separation of roles between the PoA governance layer (admission control) and the Layer 1 consensus layer — which still relies on PoH for time ordering and Tower BFT for voting and announcing finality — is shown in รูปที่ 5 [26].



รูปที่ 5. Separation of concerns in the consortium network (illustrative): the PoA layer governs admission control over an authorized validator set, while Layer 1 consensus is unchanged — PoH provides verifiable time ordering and Tower BFT provides stake-weighted voting and finality. Roles, not measured throughput.

At this network level, at least five items must be defined before real deployment, namely the number of validators and the entities owning the nodes, the method of binding identity to a validator key, the policy for adding or removing validators, the process for upgrading programs or the genesis/configuration, and the acceptable fault model. This article therefore treats the PoA Solana-compatible network as an architectural assumption of the simulation system, not as a conclusion that Solana's public-network consensus has been modified.

E. Smart Contract Programs

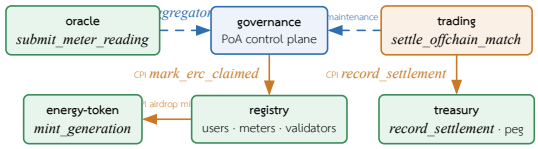
The Smart Contract is divided into several programs by responsibility, namely registry, trading, oracle, energy-token, governance, and treasury, as well as programs for performance benchmarking (blockbench and tpc-benchmark), developed with the Anchor Framework to systematically define account validation, instruction handlers, and the event log. The registry program has register_user and register_meter for registering users and Smart Meters. The trading program has create_sell_order and create_buy_order (as well as submit_limit_order and submit_market_order) for submitting sell and buy energy orders, match_orders and clear_auction for matching the trade orders proposed by the Backend, and settle_offchain_match and execute_atomic_settlement for settling transactions as an atomic settlement, where settle_offchain_match verifies the Ed25519 signatures of both buyer and seller and uses an order nullifier to prevent replay. The energy-token program has mint_generation and mint_to_wallet for creating energy tokens that must be co-signed by the REC certifying authority (Renewable Energy Certificate), where mint_generation uses the key (meter_id, settlement window) to guarantee idempotency per time window, and burn_tokens for burning energy tokens through SPL instructions. In addition, the oracle program has submit_meter_reading for receiving readings from the AMI, trigger_market_clearing for setting the boundary of the 15-minute time window (900 seconds), and aggregate_readings for aggregating the counters of readings that pass and fail validation. The timeline of the aggregation window and market clearing is shown in รูปที่ 7. The governance program acts as a PoA control layer for issuing and revoking Renewable Energy Certificates (REC), governing authority permissions, managing the aggregator allow-list, and DAO voting (the account structure and control-layer roles are described in หัวข้อ VI.E.2; the network-governance details are in the Consortium Network section). The treasury program manages staking of GRX tokens, reward payouts, and maintaining the peg of the THBC stablecoin through instructions such as stake_grx, unstake_grx, claim_rewards, swap_grx_for_thbc, redeem_thbc_for_grx, and record_settlement, which is called via Cross-Program Invocation (CPI) from the trading program. The blockbench and tpc-benchmark programs are used to run standard workloads, namely the BlockBench microbenchmark, YCSB, SmallBank, and TPC-C, for performance evaluation on the Solana-compatible network [25], [35]. The on-chain program IDs of the six core programs, declared with *declare_id!*, are summarized in ตารางที่ VI.

ตารางที่ VI
On-chain program identities (Anchor *declare_id!*) of the six core programs on the Solana-compatible consortium network. These deterministic addresses pin the deployed artifact for reproducibility.

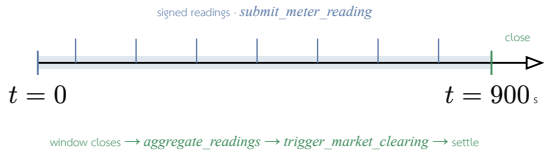
Program	Program ID (`declare_id!`)
registry	`FcSd5x4X1nzJMKLZC4tMZXnQ1ipLrGsEfeoH8N4mvJX7`
trading	`CnWDEUhTvSixeLSyViWgAnnu9YouBAYVGcrrFm1s9WcX`
oracle	`64Vgos61STZ8pW9NnHi2iGtXMTQr7NqBoMorK6Zg8RJU`
energy-token	`6FZKcVKCLFSNLMxypFJGU4K14xUBnxNW9VAuKGhmqjGX`
governance	`FokVuBSPXP11aeL7VZWd8n8aVAhWqVpyPZETToSxdvTS`
treasury	`FfxSQYKUmx9NGdCC9TDPmZSYjWYE1h4ruu3JatzHN5Tn`

The account structure uses Program Derived Addresses (PDA) to separate the system state into sub-accounts keyed by specific seeds, such as registry and user account, meter account, market and market_shard, order and trade record, escrow, order nullifier for replay prevention, oracle_data, and mint authority (gen_mint, thbc_mint). Using PDAs allows the program to verify the ownership

and deterministic address of an account without relying on the private key of each state account. Resource-intensive work — such as complex price computation, matching large order books, or grid-stability analysis — is therefore offloaded to the Backend and the Aggregator Bridge before the verified results are submitted to the Smart Contract, so as to stay within the compute constraints of a transaction [27], [36]. The order-matching mechanism in the Trading Service uses a Continuous Double Auction (CDA) algorithm with price-time priority that partitions the order book by zone and accounts for energy-flow constraints (wheeling and loss) before sending the matched order pairs for atomic settlement on the Smart Contract. The pricing and fee formulas are described in หัวข้อ V.A. The relationships among the six programs are of two kinds: Cross-Program Invocations (CPI) that write state, and control-plane reads in which one program reads another program's account to determine operation rights without invoking a CPI, as summarized in รูปที่ 6.



รูปที่ 6. Inter-program relationships among the six Anchor programs. Solid arrows are Cross-Program Invocations that write state (governance⟶registry on ERC issue, registry⟶energy-token on airdrop, trading⟶treasury on settlement). Dashed arrows are control-plane reads with no CPI: trading reads ***governance*** for the maintenance gate and ERC validity, and oracle validates an admitted-aggregator entry before accepting a reading. Governance (blue) is the policy source; trading (orange) initiates on-chain settlement.



รูปที่ 7. Market-clearing timeline: signed meter readings ingest continuously over the 15-minute (900 s) aggregation window; at window close the oracle aggregates readings, triggers clearing, and the matched pair is settled on-chain.

1) Trading: On-chain Order Lifecycle and Escrow

The trading program manages the lifecycle of trade orders and the on-chain escrow accounts, complementing the settlement path in หัวข้อ IV. Users create orders via create_sell_order and create_buy_order (as well as submit_limit_order and submit_market_order), which create a per-order PDA Order account bound to the owner, with the order expiry (expires_at) set to 24 hours (86,400 seconds) from the creation time. A sell order must reference an ERC certificate that has not yet expired before it can be created, consistent with the REC governance in หัวข้อ VI.E.2. Before entering the market, the user deposits tokens into the escrow account via deposit_escrow and withdraws the unused portion via withdraw_escrow, where the escrow account is an SPL token account under a PDA signed by the market_authority.

Order matching has two complementary paths. The first path is the off-chain matching engine (trading-engine) that runs CDA with price-time priority over the whole zone's order book at high throughput (see หัวข้อ V.C and the measured rate in หัวข้อ VIII.D). The second path is the on-chain match_orders instruction that verifies and records each matched pair, touching only the two Order accounts and the zone_market, then writing a trade record without moving tokens, hence with a low compute cost (see รูปที่ 11). The cancel_order instruction removes an order still pending in the book.

Market-level accounts are split into per-zone zone_market accounts that store order book depth, transmission capacity, and committed_flow, separated from the central Market account, so that order submission and the enforcement of cross-zone flow caps can run in parallel under the Sealevel model (see หัวข้อ VI.E.7). Once an order pair has been matched, it is sent for atomic settlement via settle_offchain_match according to the settlement model in หัวข้อ IV.

2) Governance Program: On-chain PoA Control Plane

The governance program acts as a Proof of Authority control plane on-chain, designed as a single program that consolidates 20 instructions divided by function into five subsystems (19 main instructions, together with one statistics-query instruction get_governance_stats). The key architectural point is that governance does not operate in isolation but is the source of truth from which other on-chain programs read state to determine their own behavior; that is, governance records the "policy" while trading and oracle are the ones that "enforce" that policy at the boundary of their own program. The five subsystems comprise:

- Authority system: initialize_governance creates the initial singleton account, and authority transfer is done in two steps, propose_authority_change ⟶ approve_authority_change, where the recipient must sign themselves, together with cancel_authority_change and a 48-hour expiry. There is no single-step authority-transfer path (further described in หัวข้อ VI.D).
- Config gates: update_governance_config (enabling/disabling the ERC check and transfers), update_erc_limits, set_maintenance_mode (the system-wide stop switch), update_authority_info, and set_oracle_authority. Every instruction must be signed by the authority.
- ERC certificates: issue_erc validates against the policy in GovernanceConfig then invokes a Cross-Program Invocation (CPI) to the registry to mark the energy as claimed (mark_erc_claimed), together with validate_erc_for_trading, transfer_erc, and revoke_erc.
- DAO voting system: create_proposal ⟶ cast_vote (weighted by generation) ⟶ execute_proposal, restricted to modifying only the defined parameters of the ZoneConfig, without touching the PoA authority (the details of weighting and quorum are in หัวข้อ VI.D).
- Aggregator allow-list system: admit_aggregator and revoke_aggregator (authority only), where each entry is a dedicated PDA account that performs the admission of an off-chain validator node one at a time.

The program's state accounts are PDA accounts whose addresses are deterministically derived from specific seeds, as summarized in ตารางที่ VII.

ตารางที่ VII

State accounts (PDA) of the governance program: seeds and stored data. The GovernanceConfig account is a singleton with a fixed size of 405 bytes to support upgrades, while the remaining accounts are created one entry per entity (per-cert / per-node / per-zone / per-proposal).

Account (PDA)	Seed	Stored data
`GovernanceConfig`	`[poa_config]`	singleton: authority, pending_authority, ERC policy, maintenance flag, min_quorum_votes, and counters
`ErcCertificate`	`[erc_certificate, cert_id]`	REC: owner, energy (kWh), status, expiry date
`AggregatorEntry`	`[aggregator, pubkey]`	admitted off-chain validator node (allow-list)
`ZoneConfig`	`[zone_config, zone_id]`	per-zone parameters: wheeling charge, incentive, loss factor
`Proposal`	`[proposal, zone, id]`	DAO proposal: target parameter, votes for/against, status, expiry time
`VoteRecord`	`[vote, proposal, voter]`	one vote per (proposal, voter) pair

The point that makes governance truly a control plane is the way other programs consume its state. The trading program reads the GovernanceConfig account on every order-creation instruction, then calls is_operational() to block operation when the system is in maintenance mode, along with checking the validity of the ERC before accepting a sell order; while the oracle program checks the AggregatorEntry account to allow only admitted nodes (or the designated chain bridge) to submit readings via submit_meter_reading. Both cases are read-only state accesses that deserialize the account themselves and check the owner and the PDA address without invoking a CPI, which avoids the cost of a CPI and reduces policy enforcement (gating) to merely checking the accounts submitted within that transaction. In this sense the AggregatorEntry account is the on-chain record of the admission of an off-chain validator node, enforced for real at the point where the oracle admits a reading into the system.

This structure reflects two separate PoA layers: the operational layer that determines who is authorized to run a validator on the permissioned network, and the application layer where GovernanceConfig.authority controls the program-administration rights, with the DAO being power-bounded to adjusting only zone parameters and unable to seize the authority's power or modify the validator allow-list. Furthermore, the GovernanceConfig account is fixed at a constant size of 405 bytes with reserved space (*_reserved*) for upgrades, so that new fields can be added without migrating existing accounts — a design discipline that is necessary when other programs each rely on this account's layout to deserialize it themselves.

The lifecycle of the ERC certificate is designed around the anti-double-claim property. Before issuing a certificate, the issue_erc instruction computes the unclaimed energy as $\text{unclaimed} = \text{total\_generation} - \text{claimed\_erc\_generation} - \text{settled\_net\_generation}$ in a saturating manner, then enforces that the requested issuance amount must not exceed this value ($\text{energy\_amount} \leq \text{unclaimed}$); otherwise it is rejected. It also checks the policy from GovernanceConfig via can_issue_erc() (the system must be operational and have the ERC check enabled, and the amount must be within the range of min_energy to max_erc). When the conditions are met, the program invokes a CPI to the registry to increment the claimed-energy counter (mark_erc_claimed), so that the same unit of energy cannot be certified twice. A newly issued certificate has the status Valid but sets validated_for_trading to false until it passes the validate_erc_for_trading instruction (it must be operational, in the Valid status, and not yet expired), which is a mandatory condition before a certificate can back a trade order. The revoke_erc and transfer_erc instructions control revocation (preventing double revocation) and transfer of rights (allowed only when certificate transfer is enabled or it is a transfer from the issuer itself), with every handler updating the aggregate counters in GovernanceConfig (the number of certificates issued/validated/revoked and the cumulative energy certified).

The DAO proposal lifecycle has several layers of guards beyond the generation-weighted vote (described in หัวข้อ VI.D). The create_proposal instruction rejects a non-positive voting period and enforces that the proposer be the owner of the referenced meter, by reading the MeterAccount account zero-copy then checking that meter.owner matches the proposer. The cast_vote instruction is accepted only when the proposal is in the Active status and the time has not yet passed expires_at, otherwise it is rejected with ProposalExpired, and the voter must likewise be the owner of a meter. Double-voting is prevented with a VoteRecord PDA account per (proposal, voter) pair, where the second creation of the account fails because the account already exists. The execute_proposal instruction enforces that the time has passed expires_at first, then tallies the result automatically: if the total votes are below quorum (min_quorum_votes) the proposal is rejected, while if quorum is reached and the votes in favor exceed those against the proposal passes (a tie counts as rejected); and when it passes it may adjust only four ZoneConfig parameters, namely the incentive multiplier, wheeling charge, loss factor (which must be positive), and maintenance mode, reaffirming the DAO's bounded authority.

The maintenance mechanism exhibits the property of a system-wide kill switch through a single-point account write. When the authority calls set_maintenance_mode(true), a single boolean value on the singleton GovernanceConfig account is flipped, causing every order-creation instruction in the trading program in every zone to be rejected immediately with MaintenanceMode, without redeploying the trading program. The read on the trading side skips the first 8-byte discriminator of the account then decodes Borsh directly according to the struct layout, so the gate is tolerant to changes in the governance account's discriminator as long as the field order does not change, consistent with the fixed account size and reserved space mentioned above.

3) Registry: Validator Registration and Bond

The registry program is the central registry of users, meters, and validators. The register_user and register_meter instructions create UserAccount and MeterAccount accounts zero-copy, bound to the owner via a PDA. Beyond its registry role, this program also holds the validators' security bond: register_validator requires staking a minimum of 10,000 GRX (MIN_VALIDATOR_STAKE) before being granted validator status, while stake_grx transfers GRX into the central treasury with a 24-hour withdrawal cooldown. When a validator misbehaves, the slash_validator instruction, signed by the PoA authority, slashes the bond proportionally to the severity, namely $\text{slash} = \lfloor \text{bond} \times \text{slash\_bps} \ / 10000 \rfloor$, paying compensation to the injured party not exceeding the proven loss ($\text{compensation} = \min(\text{slash}, \text{proven\_loss})$), with the remainder going to the slash fund to remove the incentive for accusations made in hope of reward. A full slash changes the status to Slashed (permanent, re-registration prohibited), while a partial slash that drops below the

minimum changes it to Suspended (recoverable by topping up the bond). This stake-and-slash mechanism is an economic incentive that complements the PoA admission control. In addition, the initial grant of 10 GRX to new users is separated out of register_user into a distinct claim_airdrop instruction (which invokes a CPI to energy-token to mint, signed by the registry's PDA) so that registration does not fail the whole transaction if the mint has a problem.

4) Oracle: Parallel Reading Ingest and Market Clearing

The oracle program ingests meter readings on-chain via submit_meter_reading, designed to write to a per-meter MeterState account (seed *[meter, meter_id]*) while the central OracleData account is read-only. Readings from different meters therefore have non-overlapping write sets and can be processed in parallel under the Sealevel model. However, the code states the real-world constraint that parallelism occurs only when the signers paying the fee (the fee payers) differ; if multiple readings use the same gateway signer, those entries are still processed sequentially because the payer account is write-locked. Before accepting a reading, the program checks for anomalies by checking the energy-value range (min/max) and the production-to-consumption ratio, computed as integers to avoid floating-point numbers ($\text{produced} \times 100 \leq \text{max\_ratio} \times \text{consumed}$). The right to submit readings is limited to the designated chain bridge, or to nodes in the governance allow-list, via the AggregatorEntry account check in authorize_node_caller (see หัวข้อ VI.E.2). Market clearing is done via trigger_market_clearing, which enforces that the epoch window boundary align to 900 seconds (*epoch_timestamp % 900 == 0*) and records last_cleared_epoch to prevent re-clearing or going back in time.

5) Energy Token: Idempotent Minting and REC Governance

The energy-token program manages the minting of energy tokens under the governance of the REC certifying committee. All three minting paths (mint_to_wallet, mint_generation, mint_tokens_direct) require the signer to be in the set of REC certifiers (up to 5) once certifiers have been set (*rec_validators_count > 0*), making it a co-signature of the renewable-energy certifying authority before minting tokens. The mint_generation instruction, which creates tokens from actual generation, guarantees idempotency per time window using a GenerationMintRecord account per (meter_id, window_start_ms) pair, created with init_if_needed and enforced to align the window boundary to 900,000 milliseconds, setting the minted flag to true only after a successful mint. A repeated call at the same window therefore returns a no-op, and if the mint fails the flag remains false so it can be retried. The mint_tokens_direct instruction is the CPI target that the registry invokes to grant the initial tokens, while burn_tokens burns tokens through the SPL Token-2022 instruction. The TokenInfo account, which stores the certifier set and counters, is zero-copy so that the high-frequency path can read it without a write lock.

6) Treasury: Settlement Recording and THBC Peg

The treasury program is the CPI endpoint for recording settlements from the trading program and is the financial core of the system. Batch settlement recording (record_settlement_batch) writes a SettlementRecord account per (zone_id, batch_id) pair that stores the merkle root (merkle_root, 32 bytes), the total value, and the value-added tax (vat_amount, vat_rate_bps) as a commitment for retrospective auditing, where the merkle root binds the set of transactions in the batch on-chain while the leaves of the tree are stored off-chain. This base instruction reconciles the central total (total_settled_thbc) directly. To support a large number of concurrent settlements without the treasury account becoming a bottleneck, there is a parallel instruction (record_settlement_batch_sharded) that records the SettlementRecord as before but adds the amount to a per-shard accumulator account instead of the central total, partitioned into 16 shards (*settle_shard_for(key) = key[0] % 16*), then reconciled into the center afterward with aggregate_settlement_shards. On the THBC stablecoin peg, the swap_grx_for_thbc instruction enforces that the new supply must not exceed the attested reserve (*new_supply <= attested_reserve*) and that the attestation has not yet expired (*now - attestation_ts <= attestation_ttl*), while redeem_thbc_for_grx limits redemption so that it does not exceed the collateral in the vault (*grx_out <= swap_vault.amount*). In addition, staking GRX to earn rewards uses a MasterChef-style accumulator (acc_reward_per_share scaled by 10^{12}) that increases according to the rewards added ($\text{delta} = \text{amount} \times \text{ACC} / \text{total\_staked}$), and when a slash occurs the bond is redistributed to the remaining stakers through an increase of the same accumulator.

7) Parallelism via Sealevel and Sharding

A design pattern that recurs across several programs is the use of a per-entity PDA account, so that unrelated transactions have non-overlapping write sets and can be processed in parallel under Solana's Sealevel model. Examples include the per-meter MeterState account in the oracle program, the per-order Order and order nullifier accounts in the trading program, and the per-user escrow account. For aggregate counters that would become a bottleneck if written to a single account (such as the cumulative user count, or the cumulative settled total), the system uses sharding into 16 shards deterministically derived from the first byte of the key (*key.to_bytes()[0] % 16*), both in the registry program (users and meters) and the treasury (settled amounts), and then reconciles into the central counter afterward with administrator-level instructions (aggregate_shards, aggregate_settlement_shards, and aggregate_readings) that prevent double-counting with a bitmask. As a result, central accounts such as GovernanceConfig, OracleData, and Market are designed to be readable in a stale manner on high-frequency paths, accepting that the aggregate value lags slightly in exchange for parallelizable throughput. This design is consistent with the slot-time target near 400 milliseconds referenced in หัวข้อ VI.D. The actual on-chain transaction throughput is measured on a single validator and reported in หัวข้อ VIII.F, while the figures on a multi-node permissioned network that include the consensus cost remain future work (see หัวข้อ VIII.E).

F. Data Model and Trust Boundary

The core data of a trade order consists of order_id, user_id (the prosumer or consumer role), meter_id, side (offer or bid), energy_amount (quantity_kwh), price_per_kwh, status, expires_at, epoch_id, and zone_id, where the energy_amount must not exceed the available_surplus_kwh that the Aggregator Bridge certifies for the seller in the same time period — a condition checked in the off-chain layer. Replay prevention uses an on-chain nullifier account (order nullifier) rather than storing a nonce in the order itself. The settlement record consists of trade_id, the buy_order_id/sell_order_id pair, the cleared

energy_amount, price, fee_amount/net_amount, wheeling_charge, loss_factor, erc_certificate_id, blockchain_tx, and the settlement timestamps (created_at/confirmed_at). The escrow state is stored separately as its own PDA account (an SPL token account under the seed *escrow*), not within the settlement record.

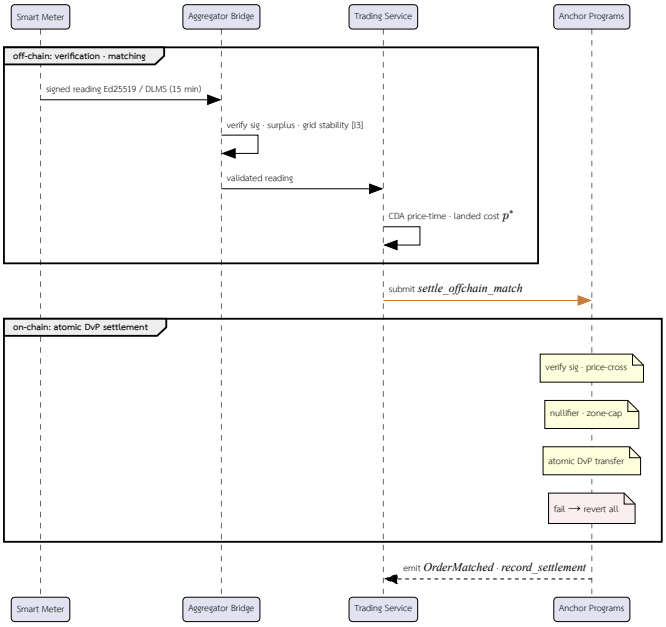
The auditability that enables the blockchain to serve as an audit layer comes from the event log emitted by the programs at every important step of the trading cycle, namely order creation (SellOrderCreated, BuyOrderCreated), matching and settlement via the OrderMatched event that records the order pair, the buyer and seller, the quantity, price, total value, and fee, emitted from both settle_offchain_match and batch_settle_offchain_match, order cancellation (OrderCancelled), bond deposit (EscrowDeposited), and market clearing (AuctionCleared). Batch settlement recording in the treasury layer emits the SettlementBatchRecorded event that binds the batch's merkle root on-chain. These events are append-only records not modified retrospectively, which external auditors use to assemble an auditable transaction history without having to trust the off-chain layer directly, consistent with the settlement and audit layer role described in หัวข้อ IV.

The boundaries of responsibility are defined clearly as follows: the Backend and the Aggregator Bridge are the ones that assess the generation and consumption data, the grid-stability conditions, Islanding safety, network constraints, and the condition that the kWh quantity does not exceed the certified value (available_surplus); then the order pairs that pass the conditions are sent to the on-chain settlement stage. The Anchor program checks only account ownership, the signer, the Ed25519 signatures of both buyer and seller on the order payload, timestamp validity (expires_at), replay prevention via the order nullifier account, the slippage/zone-capacity conditions, the escrow state, and the order/trade state not yet reused. That is, the conditions on energy quantity and oracle attestation are enforced in the off-chain layer before submission and are not re-checked on-chain. Therefore, in this prototype the blockchain serves as a settlement and audit layer, not as a full power-flow or grid-stability engine.

G. Trade Lifecycle Sequence

The trading sequence is clearly scoped between off-chain and on-chain, as in รูปที่ 8. The off-chain side is responsible for collecting Smart Meter data, authentication, verifying the reference time, assessing the grid-stability conditions, and computing the order pairs proposed for clearing. The on-chain side is responsible only for confirming the account state, the Ed25519 signatures of the buyer and seller on the signed order payload, the bounded matched pair, the escrow, and recording the settlement event.

This process makes energy transactions transparent and auditable while reducing the risk of bringing unverified Smart Meter data directly onto the blockchain. The separation of the boundary between off-chain verification and on-chain settlement is therefore a key principle in preserving both system performance and the safety of the microgrid. The limitation of this approach is that users must trust the Aggregator Bridge and the governance of the oracle_authority; to further reduce trust, one should add multi-oracle attestation or cryptographic proofs in the future.



รูปที่ 8. Trade lifecycle as a sequence diagram: off-chain participants (Smart Meter ⟶ Aggregator Bridge ⟶ Trading Service) verify and match, then submit to the on-chain Anchor Programs (via the Chain Bridge) which settle and audit. Notes tag the on-chain checks enforced during settlement (described in หัวข้อ IV); any failed CPI reverts the whole atomic DvP settlement.

VII. EXPERIMENTAL SETUP

This section summarizes the implementation details, the version of the artifact, the workload parameters, and the definitions of the metrics, so that the evaluation in หัวข้อ VIII is reproducible. We state the remaining reproducibility limitations candidly at the end of this section.

A. Implementation and Artifact

All backend services are developed in Rust (Axum) on the Tokio async runtime and communicate with one another over ConnectRPC on top of mTLS, while the path that writes data to the blockchain is decoupled through NATS JetStream [24]. The grid and Smart Meter simulation suite is developed in Python 3.11 [28] (tool details are in หัวข้อ VI.C). The whole system is arranged as a superproject that incorporates each service as a git submodule, which makes the version of the artifact identifiable from the commit of the superproject together with the pointer of each submodule. The experiments in this article refer to the state of the superproject at commit ***4b05661***, which pins the pointers of the core services — namely Aggregator Bridge, Chain Bridge, Anchor programs, blockchain-core, and Smart Meter Simulator. The on-chain cost and throughput measurements (หัวข้อ VIII.E, หัวข้อ VIII.F, and หัวข้อ VIII.G) are carried out on the gridtokenx-anchor benchmark suite at commit ***58cfc79***, which is an ancestor of the pointer pinned by the superproject ***4b05661*** and therefore lies on the same line of development.

B. Topology and Endpoints

On the evaluated path, the Smart Meter Simulator sends readings into the Aggregator Bridge through the IoT gateway (HTTP) and a gRPC channel for ingest. The Aggregator Bridge then verifies the signatures and disseminates the validated readings into zone-partitioned Redis Streams, while the chain-write path is routed

through the Chain Bridge using the NATS *chain.tx.** family of subjects (e.g., *chain.tx.submit* and *chain.tx.mint*). Separating the ports and channels in this way allows the ingest path to scale independently of the settlement path.

C. Workload Parameters

The workload in the evaluation is defined by 80 Smart Meters (*NUM_METERS=80*) that send readings every 15 seconds of simulated time (*SIMULATION_INTERVAL=15*), following the default of the simulation suite. A transmission cycle of every 15 seconds of simulated time corresponds to a time compression of approximately 60× relative to the 15-minute (900-second) window of the real meters' transmission cycle. Here we define a "reading" — the unit used to measure throughput — to mean one meter reading signed with Ed25519 that has passed signature verification and been disseminated into a Redis Stream at the Aggregator Bridge, not a matched trading order or an on-chain settlement transaction. The main workload parameters are summarized in ตารางที่ VIII.

ตารางที่ VIII
Workload parameters for the telemetry-ingest evaluation.

Parameter	Value	Meaning
NUM_METERS	80	Number of Smart Meters in the simulation
SIMULATION_INTERVAL	15 s	Reading transmission cycle in simulated time
Time compression	≈ 60×	Relative to the real 900 s window
Nominal ingest rate	5.33 readings/s	80 ÷ 15 in simulated time
Run duration	≈ 27 min	Wall-clock time of one run
Total readings	26,240	Signature verification succeeded, no loss

D. Metrics

The primary metric of the ingest-path evaluation is the ingest rate, under two definitions: the design-time rate in simulated time (nominal), equal to $80 \div 15 = 5.33$ readings per second, and the rate measured from wall-clock time, which is higher because the simulator's transmission cycle is accelerated beyond the 15-second interval of simulated time, together with the cumulative number of successfully verified readings and the data-loss ratio. The detailed measurement results are in หัวข้อ VIII.B.

E. Reproducibility Limitations

The long-running ingest-path experiment in หัวข้อ VIII.B is a single real run, while the step-load experiment in หัวข้อ VIII.C is repeated 5 times per fleet size and reports the mean together with the standard deviation on the recorded Apple M2 (8 cores, 16 GiB memory) machine. The remaining reproducibility limitation is that the workload is generated from a single sender client (parallel senders have not yet been tested), so the reported ingest-rate figures are a lower bound of the ingest path rather than the ceiling of its maximum capacity, and the on-chain settlement-path measurement is still limited to the compute-unit cost per instruction, without yet covering end-to-end latency and throughput. The next experiment should therefore add parallel senders to find the true ceiling, record the validator configuration, and extend the measurement to cover the on-chain settlement path end-to-end (see หัวข้อ IX).

VIII. EVALUATION

This section presents an architectural evaluation of the prototype system, covering the telemetry ingest rate, the off-chain order-matching rate, and the cost and throughput of on-chain transaction settlement, in order to reflect the suitability of the architecture for Peer-to-Peer energy trading at the microgrid level. In the Performance dimension, this section reports measurements of the ingest rate along the telemetry ingest path, both from a single real run (see หัวข้อ VIII.B) and from a step-load experiment that measures the loss ratio and single-sender cost (see หัวข้อ VIII.C). It also reports the on-chain processing cost of the settlement path directly, expressed as compute-units per settled order pair (see หัวข้อ VIII.E). In addition, it reports on-chain transaction throughput using standard database benchmark workloads (Blockbench, TPC-C, and SmallBank ported onto the Solana-compatible network) together with measurements of the real settlement path in หัวข้อ VIII.F. The evaluation is a characterization of a single system, not a head-to-head quantitative comparison against other platforms. Measuring the end-to-end latency and throughput of the settlement path, as well as field-level testing of the grid-stability model, remains future work.

A. Performance

The design results indicate that the Microservice architecture can structurally reduce the coupling of workloads among Smart Meter data ingest, order validation, and transaction submission to the blockchain. Separating aggregator-bridge, trading-service, chain-bridge, and settlement-engine gives each service a clear scaling point according to its workload type, particularly during periods with large volumes of Smart Meter data or with concurrent trading orders.

In terms of real-time operation, the system uses ConnectRPC and NATS JetStream [24] to separate latency-sensitive work from work that must wait for blockchain transaction confirmation. Therefore, the Backend Service is designed to respond to requests from users and Smart Meters without having to wait for transactions on the Solana-compatible network to complete immediately. This approach is expected to reduce the impact of blockchain network latency, but it still needs to be tested with a workload that explicitly specifies the number of meters, sampling interval, queue depth, and transaction mix in future work.

B. Telemetry Ingest Throughput

To evaluate the capability of the ingest path under high load, we ran a simulation with the workload in ตารางที่ VIII (80 Smart Meters, each sending a reading every 15 seconds of simulated time), using the definition of a "reading" as the unit of throughput per หัวข้อ VII (an Ed25519-signed reading that has passed signature verification and been disseminated into the Redis Stream, not a matching order or an on-chain settlement transaction).

The nominal ingest rate in simulated time equals 80 ÷ 15 = 5.33 readings per second, which corresponds to a time compression of approximately 60× relative to the reporting cycle of real meters that use a 15-minute (900-second) window. From a single real run of approximately 27 minutes, the Aggregator Bridge received and verified the signatures of 26,240 readings in total from 80 meters continuously, with no data loss. The rate measured by wall-clock time was approximately 16 readings per second, which is higher than the nominal rate because the simulator's reporting cycle was accelerated to be faster than the 15-second interval of simulated time.

This result shows that the ingest path can handle the reporting load of 80 meters stably. However, this measurement covers only the meter data ingest and verification path; it does

not yet include measurements of the throughput and latency of the on-chain settlement path, such as the throughput and latency of order matching and recording transactions on the Smart Contract. Furthermore, this experiment did not record the hardware and validator mode, so the next experiment should specify a reproducible workload, hardware profile, and validator configuration, together with measuring the settlement path end-to-end.

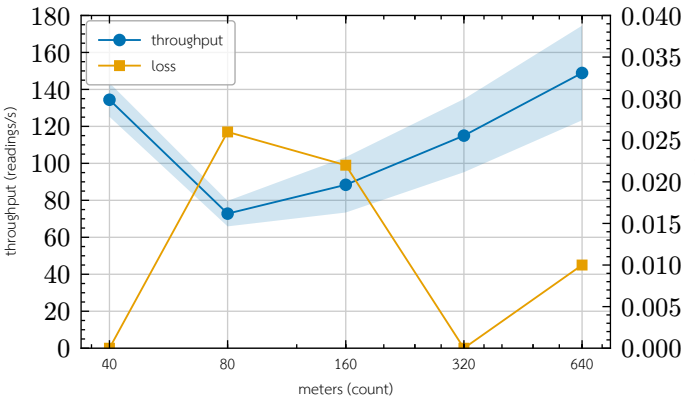
The loss ratio and single-sender cost under step-load are reported next in หัวข้อ VIII.C.

C. *Ingest Loss and Single-Sender Cost Under Step Load*

To evaluate the loss behavior of the ingest path beyond the design rate of 5.33 readings per second from หัวข้อ VIII.B, and to characterize the cost of a single full-rate sender, we apply a stepwise load ramp with meter-fleet sizes of 40, 80, 160, 320, and 640 meters, where each step runs for 60 seconds and is repeated 5 times to report the mean and standard deviation. In each run the simulator sends Ed25519-signed readings continuously at maximum rate (no delay between send rounds). The primary metric, measured at the HTTP boundary, is the fraction of requests that the Aggregator Bridge accepts, signature-verifies, and disseminates successfully into the Redis Stream (HTTP 200), where throughput = number of successful readings ÷ elapsed time and loss = number of failed requests ÷ total requests. The results are summarized in ตารางที่ IX and the trend is plotted in รูปที่ 9. The experiment runs on an Apple M2 machine (8 cores, 16 GiB memory).

ตารางที่ IX
Ingest throughput and loss vs. meter-fleet size (mean ± sd over 5 runs, 60 s each).

meters (count)	throughput (readings/s)	loss (max)
40	134.4 ± 9.2	0
80	72.8 ± 6.9	2.6×10^{-4}
160	88.4 ± 15.0	2.2×10^{-4}
320	115.0 ± 19.8	0
640	148.9 ± 25.6	1.0×10^{-4}



รูปที่ 9. Request loss (right axis) and single-sender throughput (left axis, mean ± sd) vs. meter-fleet size. The headline result is loss: it stays ≤ 0.03% across the whole tested range. Throughput is non-monotonic and single-client sender-bound — the cost of one full-rate sender, a lower bound, not a service-saturation curve.

The measurements yield two main observations. First, the loss ratio stays near zero across all fleet sizes, with the maximum measured value being approximately 2.6×10^{-4} (about 0.03%) at the 80-meter size, and several runs exhibit no loss at all. This indicates that the ingest path maintains a near-zero loss ratio (≤ 0.03%) up to a load of 640 meters. Second, the measured ingest throughput does not increase monotonically with the number of meters but fluctuates in the range of approximately 73–149 readings per second (mean), with a relatively high standard deviation (up to about 26 readings per second at 640 meters). Because this experiment generates load from a single sender client that operates asynchronously and sends per-meter readings sequentially per tick, this fluctuation and non-monotonicity likely reflect sender-side limits (such as the cost of building and signing readings, and single-client processing) together with the cadence of asynchronous dissemination into Redis, rather than saturation of the ingest service. We do not draw a definitive conclusion about the cause of this pattern from the available data.

From these results, the highest measured mean rate (mean over 5 runs at the 640-meter fleet) is approximately 149 readings per second, which is several times higher than the design rate of 5.33 readings per second and occurs at the largest fleet size tested (640 meters). However, the measured rate does not increase monotonically with the number of meters (the mean at 80 meters is lower than at 40 meters), so we do not conclude whether or not the ingest path reaches saturation from this dataset. Nevertheless, because the load is generated from a single sender client that transmits sequentially per meter per tick, this figure is a lower bound, reflecting that the Aggregator Bridge can support one full-rate sender client with near-zero loss, rather than the true capacity ceiling of the ingest path. Measuring the true ceiling requires multiple parallel senders and isolating the sender-side cost (such as onboarding and signing) from the cost of the ingest service, which remains future work. We also note that the rates in this ramp experiment (73–149 readings per second) are higher than the roughly 16 readings per second wall-clock rate of a single continuous run in หัวข้อ VIII.B, because the ramp experiment sends with no delay between rounds (max rate) to find the scaling envelope, whereas the continuous run ties the send cadence to the 15-second simulated-time interval.

D. *Off-chain Matching Throughput*

Beyond the ingest path, we measure the order-matching cost of the Continuous Double Auction (CDA) matching engine in the off-chain layer with a micro-benchmark (Criterion) that calls the matching function for one match cycle over an order book of 1,000 buy orders and 1,000 sell orders (2,000 orders total), where every buy order price is set higher than every sell order price so that matching is full, yielding a maximum of approximately 1,000 order pairs per cycle. The experiment runs on the same machine (Apple M2, 8 cores, 16 GiB memory) and collects 100 samples after the warm-up period. The results are summarized in ตารางที่ X.

ตารางที่ X
In-memory CDA matching throughput (one match cycle over 1,000 bids × 1,000 asks, 100 samples).

Metric	Value
Time per 1,000 × 1,000 match cycle (median)	32.56 ms
95% confidence interval	32.28–32.75 ms
Order processing rate (≈)	6.1×10^4 orders/s
Order-pair matching rate (≈)	3.1×10^4 pairs/s

The matching engine processes one full cycle of orders on both sides in a median time of 32.56 milliseconds (95% confidence interval approximately 32.28–32.75 milliseconds), which corresponds to a processing rate of approximately 6.1×10^4 orders per second, or about 3.1×10^4 matched order pairs per second on a single thread. This value measures only in-memory matching and does not include the cost of on-chain transaction settlement, persistence, or network communication; it is therefore an upper bound on the matching rate cleanly separated from the settlement cost. This result indicates

that in the layered architecture, off-chain matching is not the system bottleneck compared to the on-chain settlement cost in หัวข้อ VIII.E. We note that single-cycle 1,000 × 1,000 matching is a synthetic workload to measure per-cycle cost, not a steady-state rate under a real order stream with diverse price and zone distributions, which is the next step.

E. On-chain Settlement Cost

Beyond the ingest path, we directly measure the on-chain processing cost of settlement by reporting the compute units (CU) actually consumed by the off-chain match settlement instruction, read from the ***computeUnitsConsumed*** value of the confirmed transaction in the escrow settlement test on solana-test-validator 3.1.10 (Agave client). This value is a deterministic on-chain cost metric independent of the validator hardware, unlike latency on a local test network, which does not reflect the design's target network. The result is summarized in ตารางที่ XI.

ตารางที่ XI
Measured compute-unit cost of the settlement instruction (1 matched order pair).

instruction	compute units
settlement per 1 order pair	96,707

The value of 96,707 CU per settlement of one order pair[1] compared with the per-instruction default compute budget of 200,000 CU and the maximum per-transaction ceiling of 1,400,000 CU indicates that each settlement uses approximately 48% of the default budget. We note that although the maximum per-transaction compute ceiling could in theory support around 14 pairs per transaction, the binding constraint is not compute but the transaction packet size of 1,232 bytes, which squeezes the number of pairs per transaction below the ceiling of 4 pairs that the ***batch_settle_offchain_match*** program allows (see หัวข้อ IV.E). Therefore, reducing the per-pair cost via batch settlement requires changing the way signatures are packed, not merely relying on the remaining compute budget. The 48% per-instruction cost level remains consistent with the design of having the blockchain serve as a low-compute settlement layer per หัวข้อ VI.

F. On-chain Transaction Throughput

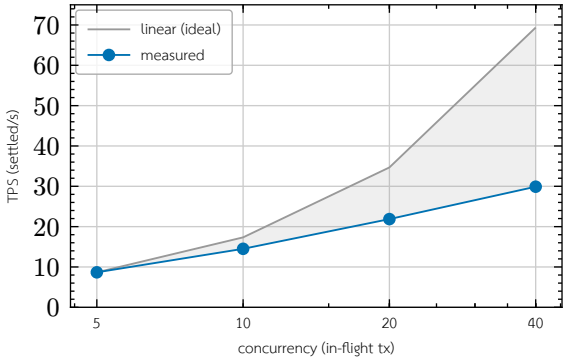
Beyond the per-instruction compute cost in หัวข้อ VIII.E, we measure the throughput of on-chain transaction processing using standard OLTP workloads ported to Solana's account model, namely BlockBench (a SIGMOD 2017-style microbenchmark together with YCSB), SmallBank, and TPC-C, augmented with a measurement of the real settlement path (settle_offchain_match). All run on a single solana-test-validator (single-node; a permissioned network governing participation via PoA) on an Apple M2 machine (8 cores) at commit ***58cfc79*** of gridtokenx-anchor, where latency is wall-clock time of the client⟶confirmed path and compute units are read from the ***computeUnitsConsumed*** of confirmed transactions. The sequential OLTP workload results are summarized in ตารางที่ XII and the TPC-C concurrency sweep results are summarized in ตารางที่ XIII.

ตารางที่ XII
Standard OLTP workloads ported to the account model (sequential, n = 150). TPS here is latency-bound by the single-client submit loop, not a throughput ceiling; ***ycsb_read*** is an RPC account fetch with no consensus round-trip.

workload · op	mean ms	TPS	CU/tx
BlockBench · ***do_nothing***	719.95	1.39	648
BlockBench · ***cpu_heavy_sort***	590.83	1.69	9,645
BlockBench · ***ycsb_read*** (RPC)	4.29	233.18	—
SmallBank · ***SendPayment***	604.63	1.65	5,963
SmallBank · ***Amalgamate***	631.62	1.58	5,936

ตารางที่ XIII
TPC-C concurrency sweep (TX = 500, 50% NewOrder / 50% Payment), 100% success at every level. Throughput from concurrent in-flight submission; CU/tx is flat across load.

concurrency	TPS	mean ms	p95 ms	CU/tx (mean)
5	8.67	575.00	726.10	21,263
10	14.50	679.34	879.95	20,633
20	21.87	856.74	1,656.15	21,269
40	29.90	1,057.79	1,707.00	21,508



รูปที่ 10. TPC-C throughput vs. concurrency on the single-node validator. Measured TPS scales sublinearly — 8× concurrency yields only 3.45× throughput — diverging from the linear-ideal reference, with the saturation knee between concurrency 10 and 20.

The most important figure for this system is the throughput of the real settlement path, not that of a generic proxy. When measuring the ***batch_settle_offchain_match*** path with an open-loop submission sweep (pre-building transactions, then firing them concurrently according to the concurrency level and re-firing dropped transactions until confirmed), we obtain a rate of only 0.5 TPS that stays flat at every concurrency level (this figure comes from a single recorded run, unlike the TPC-C/OLTP set whose raw results were committed as an artifact, so it should be interpreted as a design-indicative value pending re-measurement) (conc 5 ⟶ 0.51, conc 10 ⟶ 0.58 TPS; goodput 100%, no on-chain reverts, CU around 86–89k). It also does not scale with the number of concurrently submitted transactions; even distributing settlement across all 16 shards yields a near-identical figure (0.57–0.59 TPS), because the bottleneck is not the shard but the central set of accounts that every settlement must always write, namely the accumulator account ***total_settled_thbc*** and the fee/wheeling/loss collector accounts. Settlement is therefore global-write-bound by design; sharding can parallelize only order submission, which uses per-entity PDAs, not the reconciliation of settlement.

In the TPC-C sweep, the compute units per transaction stay flat at around 21,000 CU at every concurrency level, indicating that the throughput bottleneck is the block production time (block time around 400–600 milliseconds) together with the serialization of central account writes, not the on-chain processing. For this reason, compute units per transaction (หัวข้อ VIII.E) is a cost metric independent of workload and machine, whereas TPS is a figure that

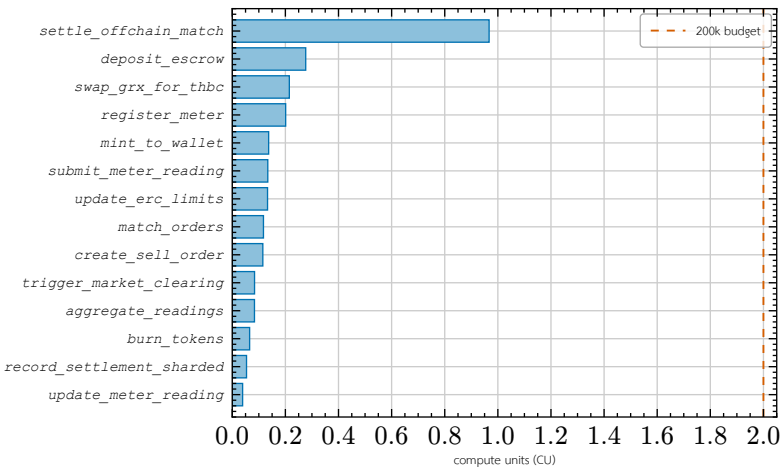
[1] The compute cost depends on the token-leg transfer pattern of the transaction. The reported value is from one escrow settlement test variant; the path with a classic-SPL to Token-2022 exchange leg measures around 103k CU, and batch settlement using Token-2022 on both legs measures around 80–92k CU. The figure of 96,707 is therefore specific to the measured variant, not a universal constant.

depends on the single validator used for testing. The sweep also exhibits sublinear scaling (concurrency 5 ⟶ 40, an 8-fold increase, yields only a 3.45-fold increase in TPS) and a saturation knee between concurrency 10 and 20 where the latency variance and p95 increase markedly, as shown against the linear reference line in รูปที่ 10.

Compared with the 900-second clearing window, even the real settlement rate of 0.5 TPS still supports around 450 settlements per window. The 80-meter workload places at most one order per meter per window, so it generates at most $\lfloor 80/2 \rfloor = 40$ matched pairs per window; the 450-settlement capacity is therefore roughly $11\times$ that demand. More generally, a single-validator settlement budget of about 450 pairs per window covers a fleet of up to roughly 900 meters at one order per meter per window before a second validator or a wider clearing window is required, which bounds the deployment size this design serves on a single node. The proxy ceiling of 29.9 TPS is equivalent to around 2.69×10^4 transactions per window. However, all results come from a single validator and should therefore be interpreted under four limitations. First, measurements on a single validator do not reflect the consensus cost (PoH together with Tower BFT) of a multi-node permissioned network, which is the basis for the observation that block time is the bottleneck. Second, BlockBench, SmallBank, and TPC-C are generic proxies, not energy workloads, so the TPS of the real settlement path is significantly lower than the proxy figures. Third, the sequential path in ตารางที่ XII at 1.4–1.7 TPS is latency-bound, not a throughput ceiling. And fourth, the TPC-C sweep TPS figures are single-point values per concurrency level, reporting confidence intervals only for latency (latency CI95) and not yet repeated over multiple runs to report a confidence interval for TPS. Multi-node open-loop measurement with repetition to find peak TPS at an SLA level is the next step (see หัวข้อ IX).

G. Per-instruction Compute-unit Profile

Beyond the cost of the main settlement path in หัวข้อ VIII.E, we measure the per-instruction compute cost of every on-chain program in the real execution path, to confirm by measurement that the on-chain compute load is kept thin by design. All values are read from the transactions' ***computeUnitsConsumed***, measured in-process with LiteSVM over a default-feature build at commit ***58cfc79*** of gridtokenx-anchor, except for the settlement instruction ***settle_offchain_match***, which is measured on a live validator (หัวข้อ VIII.E). Because compute units are deterministic, they are comparable between the two methods. The result is summarized in รูปที่ 11.



รูปที่ 11. Measured per-instruction compute-unit cost across the on-chain programs (LiteSVM ***computeUnitsConsumed***, default-feature build; ***settle_offchain_match*** measured on a live validator per หัวข้อ VIII.E), sorted by cost. Only the signature-verifying settlement path approaches half the 200,000 CU per-instruction budget (dashed); every other instruction stays well below — the measured signature of a thin settlement layer.

From รูปที่ 11, every instruction except the signature-verifying settlement path (***settle_offchain_match***) uses no more than approximately 28k CU, or about 14% of the 200k CU default budget, with the high-frequency paths at the lowest levels, namely ***update_meter_reading*** (3.9k) and ***record_settlement_sharded*** (5.4k), consistent with the design of having per-entity PDA accounts on the hot path not write-lock central accounts. This result confirms by actual measurement (not estimation) that heavy processing work — such as data-format validation, evaluation of grid-stability conditions, and queuing of buy/sell orders — is moved to the Backend layer first, while the Smart Contract enforces only the minimal conditions verifiable on-chain (account ownership, Ed25519 signatures, order nullifier, and escrow state), thereby serving as a low-compute settlement and audit layer as designed.

On the I/O and storage side, the architecture sends high-frequency meter data through the Aggregator Bridge for screening first, then records only the event log of transactions necessary for retrospective auditing onto the chain (see รูปที่ 4). Quantitative measurement of record volume, retention policy, and storage cost remains future work.

H. Fleet-Scale Solver Throughput and Live On-chain Mint Validation

Beyond the ingest-path measurements (หัวข้อ VIII.B, หัวข้อ VIII.C) and the on-chain cost/throughput results (หัวข้อ VIII.E, หัวข้อ VIII.F), we ran two further experiments to probe the simulator's capacity envelope along orthogonal axes: (a) solver and reading-generation compute cost as fleet size grows by orders of magnitude, with no network egress at all, and (b) confirmation that the surplus energy the simulator computes actually reaches a real on-chain mint through every layer of the stack. Both experiments are single exploratory runs, not repeated-trial benchmarks with reported mean and standard deviation as in หัวข้อ VIII.C, and should be read as design-indicative figures pending repeated measurement, not final reference numbers.

1) Solver Throughput at Fleet Scale

The first experiment drives ***SimulationEngine.tick()*** directly, bypassing the network path entirely (no Aggregator Bridge, no blockchain), to isolate the cost of per-meter device-model generation and the power-flow solve on the 80-bus reference topology at fleet sizes of 10,000, 50,000, and 100,000 meters. Rooftop-PV assignment is randomized per meter at a 10% ratio (rather than pinning one meter per bus as in the original default) via the ***SOLAR_PROSUMER_RATIO*** parameter. Results are summarized in ตารางที่ XIV.

ตารางที่ XIV
Per-tick solver wall-clock cost vs. fleet size (single exploratory run, 4 ticks/case, no network egress).

Meters	PV share	median tick	p95 tick	max tick
10,000	9.90%	16.5 s	17.7 s	17.7 s
50,000	9.81%	97.6 s	107.8 s	107.8 s
100,000	9.97%	230.4 s	397.5 s	397.5 s

The measured PV share converges toward the configured 10% target as fleet size grows, consistent with the law of large numbers over the per-meter random draw, and confirms that

the *SOLAR_PROSUMER_RATIO/GRID_CONSUMER_RATIO/ HYBRID_PROSUMER_RATIO* parameters now actually take effect for topology-backed fleets (prior to the fix, the split was hardcoded to 70/20/10 and these parameters were silently ignored). Per-tick wall-clock cost scales mildly super-linearly with fleet size — the 100,000-meter case takes roughly 14× the 10,000-meter case's tick time for a 10× larger fleet — indicating that per-meter reading generation (single-threaded CPU-bound work dispatched via *asyncio.to_thread*) is the bottleneck, not the power-flow solve, whose topology size is fixed regardless of meter count.

2) Live On-chain Mint Proof

The second experiment probes the opposite axis: a small fleet (100 meters) with the full network path enabled — meter-owner onboarding through the IAM Service (register ⟶ verify ⟶ login, with on-chain PDA registration), Ed25519 device-key registration in the Aggregator Bridge's device registry, mTLS- and AES-256-GCM-encrypted signed reading ingest every tick, and a real on-chain mint transaction signed and submitted by Chain Bridge once a settlement window closes. The run targets the same single solana-test-validator used in หัวข้อ VIII.F.

Results from a 100-meter, 5-tick run confirm the full path is consistent end to end: owner onboarding for all 100 meters completed in roughly 11 s; every ingested reading passed signature verification and was disseminated into its zone Redis Stream; and — the key proof point — **140 real on-chain mint transactions across 20 unique meters** were observed, each carrying a verifiable Solana transaction signature and slot number (for example, a 0.08568 kWh mint with signature *3NbmxvqEKRLfM2yEYJLNy7axVcibwBXTzVbDAPAhdbNaN6432PdVJa3mAvnxuqK6Y2dZ64BRdKK29bM1ttyjV7kw* at slot 1424). This closes the gap the first experiment deliberately leaves open — that experiment drives *tick()* directly without calling *start()*, so it never touches the Aggregator Bridge or the chain at all — by demonstrating that the net-surplus figures the simulator computes agree with what is actually minted on-chain.

This experiment is deliberately small in scale: IAM onboarding is one HTTP round-trip per meter owner, and each mint is a genuinely signed Solana transaction per settlement window. Running the 10,000–100,000-meter fleets from the solver-throughput experiment with the full network path enabled is out of scope for this work and is left for future work that scales both IAM onboarding throughput and the test validator's capacity accordingly (see หัวข้อ IX).

3) Fleet-Scale Live Mint Throughput

A third experiment closes part of the future-work gap the previous one leaves open: driving the **full** network path (IAM onboarding, signed encrypted ingest, settlement, and Chain Bridge minting) at fleet sizes in the thousands rather than 100, using a dedicated harness against the same single solana-test-validator. Each meter's one net-surplus reading is backdated into an already-closed 15-minute settlement window so the sweep evicts it without waiting for real time to pass. Results are summarized in ตารางที่ XV; as with the two experiments above, these are single exploratory runs, not repeated trials.

ตารางที่ XV

Fleet-scale live on-chain mint throughput (single exploratory runs).

Fleet	Minted	Overall TPS	Steady TPS (10 s median)	Peak TPS (10 s)
1,000	1,000 / 1,000	29.3	—	—
10,000	10,000 / 10,000[2]	18.4	37.7	123.2
25,000 (tuned)	25,000 / 25,000	23.2	14.6	193.4

At an untuned baseline, a 25,000-meter burst overloads Chain Bridge's mint-consumer queue: envelopes queue past the aggregator's 55-second staleness cap faster than the fixed-concurrency consumer (8 in-flight confirmations) can drain them, so rejected envelopes are republished, deepening the queue further — goodput decayed from 16.7 to 3.7 mints/s in a positive-feedback congestion collapse before the run was stopped manually (2,559 of 25,000 minted). Raising the mint-consumer concurrency (8 ⟶ 64), skipping a redundant pre-flight simulation, and lengthening the aggregator's reply-wait budget (30 s ⟶ 120 s) eliminated the collapse: the tuned 25,000-meter row above cleared with zero stale-rejects and zero republish amplification (versus 65,117 stale-rejects and a 22,528-entry outbox re-published every tick at baseline).

The 10,000-meter run also surfaced a genuine operational bug worth reporting on its own terms: a subset of onboarded users' wallet-link writes were lost when the IAM service itself was OOM-killed during the highest-concurrency onboarding bursts. The root cause was that IAM's request-metrics middleware labeled Prometheus counters by the literal request path rather than the route template, so each *PATCH /wallets/{id}* call — issued once per onboarded user to set a primary wallet — permanently added a new label series; resident memory therefore grew with **cumulative onboard count**, not concurrency, until the container's memory limit was exceeded. No surplus was permanently lost — the affected meters were flagged "no wallet registered, kept for retry" and the durable mint outbox re-minted them into their original settlement window once the wallet links were repaired (the 79-meter tail in ตารางที่ XV) — but the incident is a concrete illustration of why per-request metrics must be labeled by route template, not literal path, under any workload with per-entity path parameters.

4) On-chain Telemetry Write Scaling by Meter Count

A fourth experiment isolates the on-chain layer of the telemetry path — the oracle program's *submit_meter_reading* instruction alone, bypassing IAM and the Aggregator Bridge entirely — to answer a question the mint experiments above leave open: does on-chain state-write capacity degrade as the per-meter PDA account set grows into the hundreds of thousands? The harness builds client-signed transactions against a cached blockhash, submits them *skipPreflight* through a windowed open loop capped at 3,000 in-flight transactions, and confirms via bulk *getSignatureStatuses* polling (256 signatures per call at a 1.5 s cadence, so reported latencies are quantised by ±1.5 s), at fleet sizes of 10,000, 50,000, 100,000, 150,000, and 200,000 meters, two epochs each: the first epoch creates each meter's PDA (init, heavier compute), the second overwrites existing state (steady). All scales run back-to-back on the same single solana-test-validator without a ledger reset in between, so the 200,000-meter case starts against an existing set of over 310,000 meter PDAs and the validator ends the sweep holding 510,000 meter PDAs from this experiment alone; the experiment totals 1,020,000 transactions. Results are summarized in ตารางที่ XVI; as with the other experiments in this section, each scale is a single exploratory run.

[2] 9,921 minted directly within the watch window; the remaining 79 landed later via the durable mint outbox's retry — see the wallet-link incident below.

ตารางที่ XVI
On-chain telemetry write throughput vs. meter count (single exploratory run per scale; latency quantised ±1.5 s by the status-poll cadence).

Meters	ok/total	TPS (init)	TPS (steady)	p50 ms	p95 ms	CU steady (med)
10,000	19,908 / 20,000	439.2	429.1	1,546	2,354	15,025
50,000	99,524 / 100,000	319.8	243.1	1,868	3,064	13,525
100,000	199,048 / 200,000	409.6	455.5	1,553	2,346	15,060
150,000	298,572 / 300,000	449.4	426.2	1,586	2,402	13,560
200,000	398,096 / 400,000	438.5	443.2	1,582	2,391	13,560

The headline finding is flat throughput across scale: steady-state TPS stays in the 426–455 band from 10,000 through 200,000 meters (the 243 at 50,000 is a transient validator stall that does not recur at larger sizes), p50 latency holds at roughly 1.5–1.9 s, and per-transaction compute cost is constant (steady median 13,525–15,060 CU, far below the 200,000 CU per-instruction budget) independent of the accumulated account count. This empirically confirms the per-meter PDA state design — one ***MeterState*** account per meter, with the hot path never writing the global ***OracleData*** account — keeps reading writes parallelizable under Sealevel, and that the remaining serialization point is the single gateway fee-payer wallet (write-locked on every transaction), not account-set size. Every failed transaction (0.46–0.48% per scale) is a deterministic logical rejection by the oracle's own anomaly detector — a small fraction of the synthetic readings violate the production-to-consumption ratio cap, checked by integer cross-multiplication (***AnomalousReading***) — with not a single send rejection, queue overflow, or blockhash expiry at any scale. The global aggregate counters on ***OracleData*** remain zero throughout by design: the hot path treats that account as read-only, and reconciling fleet totals is the job of the periodic administrative ***aggregate_readings*** instruction. The key limitation is that all results come from a single validator and a single submitting wallet, so the measured ceiling is a property of this configuration; spreading load across multiple gateway fee-payers and measuring on a multi-node cluster remain future work (see หัวข้อ IX).

IX. DISCUSSION AND LIMITATIONS

A. Discussion

The results of the simulated system design indicate that the approach of separating the operational layers between the Smart Meter Simulator, Backend Service, and Solana Smart Contract enables the Peer-to-Peer energy trading system to have a verifiable structure with clearer points for system expansion. The Backend Service acts as the primary control layer for validating data, managing permissions, and evaluating grid-stability conditions before submitting transactions to the blockchain, so that the Smart Contract does not have to bear the entire processing burden of the microgrid system.

Using the Solana Smart Contract as a Settlement and Audit layer enhances the transparency of energy transactions, because trade orders and energy settlement states can be audited retrospectively. However, the system must still prioritize the safety of the electrical grid first. Therefore, transaction approval should not consider only the price and energy quantity, but must also take into account system stability conditions and the connection status of the devices.

The three-part evaluation supports the design hypothesis that the blockchain serves as a thin settlement layer. First, the ingest path keeps the loss ratio close to zero (≤ 0.03%) up to a load of 640 devices (see หัวข้อ VIII.C), where the measured ingest rate is a lower bound of a single sender client and does not increase monotonically with the number of meters, so no conclusion is drawn about the service saturation point; this indicates that moving the high-frequency validation and aggregation work to the off-chain layer does not become a system bottleneck under the tested load levels. Second, the measured on-chain settlement cost of 96,707 CU per order pair accounts for approximately 48% of the initial per-instruction compute budget (see หัวข้อ VIII.E), showing that once most business rules are enforced off-chain, the remaining on-chain burden is at a level that opens a compute-wise opening to combine multiple order pairs into a single transaction (batch settlement), although the actual combination is constrained by the transaction packet size before the compute ceiling (see หัวข้อ IV.E), so the signature-packing method must be adjusted to actually reduce the per-order-pair cost in future work. Third, the fact that the on-chain cost is a deterministic CU value rather than varying with public-network market fees is consistent with the governance rationale for choosing a PoA consortium network (see หัวข้อ II), where transaction costs are predictable and independent of gas volatility.

In terms of economic mechanism, separating the roles of the three token types — the energy token certified by REC for delivery, the THBC coin pegged to the baht for pricing and payment, and the GRX token for stake and governance — enables transaction settlement to be DvP, which binds energy delivery to payment in a single transaction, and grants network governance rights under a consortium framework that clearly controls REC certification and the aggregator allow-list. We have analyzed the preliminary model-derived economic implications of the net amount received by sellers and the premium relative to the feed-in tariff in หัวข้อ V.B. However, the full measurement of economic properties, such as allocative efficiency / welfare and the strategic behavior of participants, remains outside the scope of the architectural evaluation in this work.

Compared with permissioned platforms widely used in the P2P energy trading literature such as Hyperledger Fabric [15] (see ตารางที่ II), the design in this work differs architecturally in three points. First, Fabric uses an execute-order-validate processing model in which chaincode runs on endorsing peers before ordering through an orderer (Raft/BFT), whereas this work uses Solana's account model that processes transactions in parallel via Sealevel when the sets of written accounts do not overlap (see หัวข้อ VI.E.7), which facilitates scaling order submission per entity but makes settlements that reconcile a central balance bound to a central write (global-write-bound), as measured in หัวข้อ VIII.F. Second, the processing cost in this work is expressed in compute units (CU) that are deterministic and have a clear per-transaction ceiling, unlike Fabric, which has no equivalent concept of a per-transaction compute budget, making this work's on-chain cost straightforward to predict and to detect regression. Third, this work enforces the provenance check of meter readings with Ed25519 signatures and replay prevention through an order nullifier directly within the program logic, instead of relying on an external endorsement policy. This comparison is qualitative (architectural) because there is not yet a head-to-head measurement on the same energy workload, which is a quantitative comparison left open for the future.

B. Limitations

A key limitation of this work is that the system is still at the simulation level, with energy data coming from the Smart Meter

Simulator and not yet tested together with real Smart Meter devices or field electrical systems. Therefore, the results reflect the suitability of the architecture and the preliminary working process rather than quantitative performance in a real environment.

In addition, the evaluation covers only part of the entire operational path, namely measuring the ingest rate of the telemetry ingest path and the compute cost of the on-chain settlement instruction on a per-part basis, but has not yet measured the end-to-end latency from meter reading to on-chain confirmation, nor the order matching rate under multiple order-volume levels. As for the throughput of the settlement path, it was measured on a single validator (see หัวข้อ VIII.F) but does not yet cover a multi-node permissioned network that includes consensus cost, and the batch settlement discussed at the design level has not yet been evaluated as an actual experiment. In terms of trust, the energy quantity and grid-stability conditions are enforced in the off-chain layer by the Aggregator Bridge and the oracle authority, whose collusion or compromise is not yet prevented by cryptographic mechanisms (see หัวข้อ III). Likewise, the security assumption of the consensus (PoH together with Tower BFT) under a permissioned network that governs participation via PoA is also an architectural assumption that has not yet been quantitatively evaluated under validators with deviant behavior.

C. Threats to Validity

The measurement results in this work have three caveats in interpretation. First (internal validity), the workload in the ingest experiment was generated by a single sender client operating asynchronously, so the measured ingest rate is a lower bound that also reflects the sender-side limitation, not the true capacity ceiling of the ingest service (see หัวข้อ VIII.C). Second (construct validity), the design rate of 5.33 readings per second is computed from the number of meters and the transmission cycle, not the maximum measured rate, so comparing the two figures must clearly state the context of each value. Third (external validity), the compute cost of the settlement instruction was measured on a single solana-test-validator version, and that value is a logical program cost (deterministic CU) independent of hardware, but the latency and throughput on a real production network may differ. All results should therefore be interpreted as an architectural evaluation on a simulated system.

D. Future Work

Future work should develop the connection with real Smart Meters via the DLMS/COSEM standard and evaluate the entire operational path more completely, including measuring end-to-end latency and extending the settlement throughput measurement reported on a single validator in หัวข้อ VIII.F to a multi-node permissioned network that includes consensus cost, together with open-loop measurement to find the TPS ceiling at the SLA level, and experimenting with batch settlement to confirm the reduction of compute cost per order pair, as well as measuring the ceiling of the ingest path with multiple parallel senders that separate the sender-side cost from the ingest service. In terms of trust, the residual trust of the off-chain layer should be reduced with multi-oracle attestation or cryptographic proof, and the consensus of the permissioned network (governing participation via PoA) should be quantitatively evaluated under nodes with deviant behavior. In addition, models for grid stability, islanding detection, and demand response should be added so that the safety evaluation of energy trading more closely approximates real usage. The next experiment should clearly define a reproducible workload, such as the number of meters, sampling interval, transaction mix, validator count, queue depth, failure/retry policy, and market-clearing output metrics.

Beyond the above, three quantitative evaluation directions are explicitly left open, namely:

- **Allocative efficiency / welfare:** measure the gains from trade of the CDA mechanism relative to a uniform-price clearing baseline and the feed-in tariff on real matching data from a full simulated workload, in order to confirm the economic premium analyzed at the model level in หัวข้อ V.B with actual measurement instead of computation from fixed parameters.
- **Cost per trade:** convert the per-transaction compute cost into a fee in lamport units and baht at a given rate, then report the fee-to-trade-value ratio, which is a key deciding criterion for real-world adoption.
- **Liveness & baseline:** evaluate the availability of the permissioned network when some validator nodes fail (1-of-N down), and conduct a quantitative head-to-head comparison with other permissioned platforms, especially Hyperledger Fabric, on the same energy workload, in order to extend the qualitative comparison in ตารางที่ II into a directly comparable joint measurement.

X. CONCLUSION

This work presents the development of a simulation system for Peer-to-Peer solar energy trading in a microgrid, integrating a Smart Meter Simulator, a Backend/Aggregator Bridge, and a Solana-compatible Smart Contract. The system is designed to handle real-time energy data, to validate trading orders through the Backend service, and to record the results of transactions that satisfy the conditions on the blockchain in order to increase transparency and auditability.

The architectural evaluation indicates that the Microservice approach combined with transaction queuing through NATS JetStream has the potential to reduce the coupling of workloads among data ingestion, transaction validation, and recording results on the blockchain. In addition, restricting the role of the Smart Contract to a thin settlement and audit layer helps to more clearly bound the on-chain compute. The evaluation covers four aspects, namely: the telemetry ingest path, which sustains the design rate of 5.33 readings per second continuously with no data loss, and which, under a step load increasing to 640 meters (averaged over 5 runs), still keeps the loss ratio below 0.03% with no saturation knee observed within the limit of the single sender client used in the test; off-chain order matching, where the CDA matching engine processes approximately 3.1×10^4 order pairs per second in memory; the on-chain settlement cost of the single-pair instruction, measured at 96,707 compute units for the reported token-leg variant (approximately 48% of the initial compute budget), which opens room for batch settlement of order pairs; and the real settlement throughput, which is global-write-bound by design — serialized on a central reconciliation write — and measures roughly 0.5 transactions per second on a single validator in a single recorded run (pending repeated measurement), yet still supports roughly 450 settlements per 900-second clearing window,

about an order of magnitude above the at most 40 matched pairs the tested 80-meter workload generates. All four results support the design that makes the blockchain a thin settlement layer. Measuring end-to-end latency, experimenting with batch settlement, and evaluating on real devices and networks are the next steps, and they form an important foundation for extending the work toward use at an industrial energy scale in the future.

REFERENCES

[1] W. Tushar, T. K. Saha, C. Yuen, D. Smith, และ H. V. Poor, "Peer-to-Peer Trading in Electricity Networks: An Overview", *IEEE Transactions on Smart Grid*, ปี 11, ฉบับที่ 4, น. 3185–3200, 2020.

[2] T. Morstyn, A. Teytelboym, และ M. D. McCulloch, "Bilateral Contract Networks for Peer-to-Peer Energy Trading", *IEEE Transactions on Smart Grid*, ปี 10, ฉบับที่ 2, น. 2026–2035, 2019.

[3] A. Paudel, K. Chaudhari, C. Long, และ H. B. Gooi, "Peer-to-Peer Energy Trading in a Prosumer-Based Community Microgrid: A Game-Theoretic Model", *IEEE Transactions on Industrial Electronics*, ปี 66, ฉบับที่ 8, น. 6087–6097, 2019.

[4] IEEE Standards Association, "IEEE Std 1547-2018: IEEE Standard for Interconnection and Interoperability of Distributed Energy Resources with Associated Electric Power Systems Interfaces". 2018.

[5] IEEE Standards Association, "IEEE Std 2030.7-2017: IEEE Standard for the Specification of Microgrid Controllers". 2018.

[6] J. Guerrero, A. C. Chapman, และ G. Verbic, "Decentralized P2P Energy Trading Under Network Constraints in a Low-Voltage Network", *IEEE Transactions on Smart Grid*, ปี 10, ฉบับที่ 5, น. 5163–5173, 2019.

[7] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System". 2008.

[8] V. Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform". 2014.

[9] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger". 2014.

[10] D. Yaga, P. Mell, N. Roby, และ K. Scarfone, "Blockchain Technology Overview", technical report NIST IR 8202, 2018. doi: 10.6028/NIST.IR.8202.

[11] E. Mengelkamp, P. Staudt, J. Garttner, และ C. Weinhardt, "Trading on Local Energy Markets: A Comparison of Market Designs and Bidding Strategies", *Applied Energy*, ปี 210, น. 870–880, 2018.

[12] E. Munsing, J. Mather, และ S. Moura, "Blockchains for Decentralized Optimization of Energy Resources in Microgrid Networks", ใน *2017 IEEE Conference on Control Technology and Applications*, 2017, น. 2164–2171.

[13] A. Esmat, M. de Vos, Y. Ghiassi-Farrokhfal, P. Palensky, และ D. Epema, "A Novel Decentralized Platform for Peer-to-Peer Energy Trading Market with Blockchain Technology", *Applied Energy*, ปี 282, น. 116123, 2021, doi: 10.1016/j.apenergy.2020.116123.

[14] J. Kang, R. Yu, X. Huang, S. Maharjan, Y. Zhang, และ E. Hossain, "Enabling Localized Peer-to-Peer Electricity Trading Among Plug-in Hybrid Electric Vehicles Using Consortium Blockchains", *IEEE Transactions on Industrial Informatics*, ปี 13, ฉบับที่ 6, น. 3154–3164, 2017.

[15] E. Androulaki *และคณะ*, "Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains", ใน *Proceedings of the Thirteenth EuroSys Conference*, 2018, น. 1–15. doi: 10.1145/3190508.3190538.

[16] L. Lamport, R. Shostak, และ M. Pease, "The Byzantine Generals Problem", *ACM Transactions on Programming Languages and Systems*, ปี 4, ฉบับที่ 3, น. 382–401, 1982, doi: 10.1145/357172.357176.

[17] M. Castro และ B. Liskov, "Practical Byzantine Fault Tolerance", ใน *Proceedings of the Third Symposium on Operating Systems Design and Implementation*, 1999, น. 173–186.

[18] M. Andoni *และคณะ*, "Blockchain Technology in the Energy Sector: A Systematic Review of Challenges and Opportunities", *Renewable and Sustainable Energy Reviews*, ปี 100, น. 143–174, 2019, doi: 10.1016/j.rser.2018.10.014.

[19] Z. Tanis, A. Durusu, และ N. Altintas, "A Comprehensive Review on Peer-to-Peer Energy Trading: Market Structure, Operational Layers, Energy Cooperatives and Multi-energy Systems", *IET Renewable Power Generation*, ปี 19, ฉบับที่ 1, 2025, doi: 10.1049/rpg2.70075.

[20] G. B. Bhavana, R. Anand, J. Ramprabhakar, V. P. Meena, V. K. Jadoun, และ F. Benedetto, "Applications of blockchain technology in peer-to-peer energy markets and green hydrogen supply chains: a topical review", *Scientific Reports*, ปี 14, ฉบับที่ 1, 2024, doi: 10.1038/s41598-024-72642-2.

[21] G. B. Bhavana, R. Anand, J. Ramprabhakar, J. M. Guerrero, N. Thakkar, และ A. Ambikapathy, "Comparative evaluation and simulation of blockchain consensus mechanisms for secure and scalable peer to peer energy trading in microgrids", *Scientific Reports*, ปี 15, ฉบับที่ 1, 2025, doi: 10.1038/s41598-025-27431-w.

[22] IEEE Standards Association, "IEEE Std 2030.8-2018: IEEE Standard for the Testing of Microgrid Controllers". 2018.

[23] SPIFFE Project, "X.509-SVID Specification". 2018.

[24] NATS.io, "JetStream". 2024.

[25] Coral, "Anchor Documentation". 2026.

[26] A. Yakovenko, "Solana: A New Architecture for a High Performance Blockchain". 2018.

[27] Solana Foundation, "Program Derived Addresses". 2026.

[28] Python Software Foundation, "Python 3.11 Documentation". 2026.

[29] D. P. Chassin, K. Schneider, และ C. Gerkensmeyer, "GridLAB-D: An Open-Source Power Systems Modeling and Simulation Environment", ใน *2008 IEEE/PES Transmission and Distribution Conference and Exposition*, 2008, น. 1–5. doi: 10.1109/TDC.2008.4517260.

[30] S. Ramirez, "FastAPI Documentation". 2026.

[31] A. A. Hagberg, D. A. Schult, และ P. J. Swart, "Exploring Network Structure, Dynamics, and Function Using NetworkX", ใน *Proc. 7th Python in Science Conference*, 2008, น. 11–15.

[32] K. S. Anderson, C. W. Hansen, W. F. Holmgren, A. R. Jensen, M. A. Mikofski, และ A. Driesse, "pvlib python: 2023 Project Update", *Journal of Open Source Software*, ปี 8, ฉบับที่ 92, น. 5994, 2023, doi: 10.21105/joss.05994.

[33] L. Thurner *และคณะ*, “pandapower: An Open-Source Python Tool for Convenient Modeling, Analysis, and Optimization of Electric Power Systems”, *IEEE Transactions on Power Systems*, ปี 33, ฉบับที่ 6, น. 6510–6521, 2018, doi: 10.1109/TPWRS.2018.2829021.

[34] S. Joshi, “Feasibility of Proof of Authority as a Consensus Protocol Model”. 2021. doi: 10.48550/arXiv.2109.02480.

[35] Solana Foundation, “Tokens on Solana”. 2026.

[36] Solana Foundation, “Solana Documentation”. 2026.